

Dossier de projet Deadline Drive



NINACH
Amir

2024



Sommaire

Compétences du référentiel couvertes	3
Présentation de l'équipe de développement	4
Présentation du développeur	5
Présentation du projet	5
a. Le projet	5
b. Gestion du projet	5
c. Spécifications fonctionnelles	7
Analyse	9
a. Diagramme de cas d'utilisation	9
b. Wireframes et maquette	9
c. Arborescence	13
Base de données	14
a. Modèle Conceptuel de Données (MCD)	14
b. Modèle Logique de Données (MLD)	15
c. Modèle Physique de Données (MPD)	18
d. Langage de Définition de Données (LDD)	20
Développement	22
a. Structure du projet	22
b. Front-end	27
c. Back-end	43
Sécurité	50
Tests	54
Déploiement	56
Recherches	57
a. Recherche en anglais	57
b. Veille technologique	57
Conclusion	58
Annexes	59
Ressources externes	70



Compétences du référentiel couvertes

Catégorie	Compétences
Développement Front-end	- Installer et configurer son environnement de travail en fonction du projet web ou web mobile
	- Maquetter des interfaces utilisateur web ou web mobile
	- Réaliser des interfaces utilisateur statiques web ou web mobiles
	- Développer la partie dynamique des interfaces utilisateur web ou web mobile
Développement Back-end	- Mettre en place une base de données relationnelles
	- Développer des composants d'accès aux données SQL
	- Développer des composants métier coté server
	- Documenter le déploiement d'une application dynamique web ou web mobile
Compétences Transversales	- Travailler en méthode agile : Application de pratiques agiles pour une gestion flexibles et réactive du projet
	- Développement de projet en équipe : Renforcement des compétences en communication et coordination
	- Adaptabilité et Résolution de Problèmes : Adaptation rapide et résolution efficace des problèmes.
	- Gestion de la Documentation Technique : Rédaction de documentations techniques claires et détaillées.
	- Veille Technologique : Maintien d'une connaissance actualisée des innovations technologiques.



Présentation de l'équipe de développement

Notre équipe est composée de quatre étudiants en Développement Web qui ont conjointement travaillé sur la conception d'une application Web fullstack. Chacun de nous a pris en charge des aspects spécifiques du développement frontend et backend, tout en contribuant à l'ensemble du projet de manière collaborative.

	<u>Calli Peter</u> : Calli a développé un calendrier interactif, intégrant des fonctionnalités qui aident les utilisateurs à suivre et à gérer leurs événements importants. Cette partie de l'application ajoute une valeur pratique significative, facilitant l'organisation personnelle des utilisateurs.
	<u>Jérôme Gavino</u> : Jérôme a développé l'interface utilisateur, en créant des pages esthétiquement plaisantes et fonctionnelles. Il a été responsable de la conception des pages de favoris et de contact, permettant aux utilisateurs de naviguer facilement et d'interagir avec l'application de manière intuitive.
	<u>Faysoil Chaambani</u> : Faysoil a pris en charge la page de citations, implémentant un système robuste d'upvote et de downvote ainsi qu'un système de création de citations. Ces fonctionnalités permettent aux utilisateurs de contribuer activement à la communauté en partageant et évaluant le contenu, enrichissant ainsi l'expérience utilisateur globale.
	<u>Amir Ninach</u> : En tant que Scrum Master, j'ai orchestré la coordination et le suivi du projet, en veillant à ce que toutes les parties du projet progressent harmonieusement. Sur le plan technique, j'ai été chargé de développer les fonctionnalités de connexion et d'inscription, ainsi que le système de commentaires et de notifications. J'ai également conçu les pages Connexion, Inscription, et À Propos, ainsi que la navbar et le footer de l'application, assurant ainsi une expérience utilisateur cohérente et intégrée.

Ensemble, nous avons formé une équipe soudée, chaque membre apportant une expertise unique et essentielle à la réussite du projet. Notre collaboration a permis de créer une application web complète et fonctionnelle qui répond aux besoins spécifiques de nos utilisateurs tout en offrant une expérience riche et engageante.



Présentation du développeur

Amir Ninach – Profil et Parcours

À 29 ans, je suis une personne profondément passionnée par les nouvelles technologies et les jeux de stratégie, des domaines qui requièrent réflexion, logique et anticipation. Mon intérêt pour ces secteurs m'a toujours poussé à chercher des solutions innovantes et à améliorer continuellement mes compétences.

Anciennement manager commercial, j'ai acquis une solide expérience dans la gestion d'équipe et la communication, compétences que j'ai trouvées extrêmement utiles dans ma transition vers le monde du développement web. Cette reconversion professionnelle n'était pas seulement un changement de carrière mais une décision mûrement réfléchie pour poursuivre ma véritable passion, la découverte de nouvelles technologies à travers le développement web.

Le développement web m'attire particulièrement pour son aspect créatif et collaboratif. J'apprécie non seulement la capacité à construire quelque chose de concret à partir de zéro, mais aussi le processus de découverte et d'apprentissage continu qu'il implique. Que ce soit en résolvant des problèmes complexes ou en travaillant en équipe pour fusionner des idées en solutions fonctionnelles, chaque projet offre de nouvelles opportunités et un nouveau défi à relever.

En tant que Scrum Master dans notre projet de fin d'année, j'ai pu mettre en pratique mes compétences en leadership en guidant notre équipe à travers les différentes phases de développement. J'ai supervisé la progression du projet en veillant à ce que chaque membre de l'équipe puisse non seulement contribuer efficacement à ce projet mais aussi se développer personnellement et professionnellement.

À travers ce parcours, je continue de chercher des moyens d'intégrer la technologie et le design de manière innovante, avec l'objectif de créer des applications web qui ne sont pas seulement fonctionnelles mais aussi esthétiquement agréables et intuitives pour les utilisateurs. Ma curiosité et ma passion pour le développement web me motivent à explorer de nouvelles technologies et à adopter des stratégies de développement avant-gardistes.

Présentation du projet

a. Le projet

Deadline Drive est une application web conçue pour offrir une gestion de l'emploi du temps de travail à la fois ludique et interactive. L'application permet aux utilisateurs de recevoir quotidiennement des notifications contenant une citation du jour sur leur compte utilisateur. Cette fonctionnalité vise à inspirer et à motiver les utilisateurs tout en leur offrant un outil pour gérer efficacement leurs délais et tâches.

Les utilisateurs ont la capacité de personnaliser leur expérience en créant leurs propres citations ou en ajoutant leurs citations favorites à la base de données de l'application. De plus, l'application intègre des fonctionnalités sociales telles que la possibilité de commenter ou de mettre en favoris les citations, ainsi qu'un système d'upvote et de downvote permettant de classer les citations tendance.

b. Gestion du projet

Gestion de projet pour le projet Deadline Drive



Le projet **Deadline Drive** a été développé en utilisant une approche agile pour maximiser l'efficacité et l'adaptabilité tout au long du cycle de développement. En tant que Scrum Master, j'ai orchestré le déroulement des sprints, assuré une communication fluide au sein de l'équipe et veiller à l'alignement des objectifs avec les attentes de notre client, Adrien Sergent.

Développement Agile et Itératif

Dès la conception initiale, notre équipe s'est concentrée sur la création de wireframes avec l'outil Figma, ce qui a permis une visualisation claire de l'interface utilisateur et a facilité les discussions avec le client pour des ajustements rapides et réactifs. Ces maquettes ont été élaborées pour adhérer strictement aux exigences du dossier de conception tout en respectant la charte graphique du projet. Nous avons intégré des considérations d'accessibilité dès le début pour garantir que l'application soit utilisable par un large éventail d'utilisateurs, y compris ceux en situation de handicap.

L'utilisation de Trello pour la gestion des tâches et le suivi des progrès a été essentielle pour maintenir le déroulement et la priorisation des activités de développement. Les daily stand-ups ont permis de résoudre rapidement les obstacles techniques et d'assurer une progression régulière du projet.

Choix Technologiques et Sécurité

Sur le plan technologique, nous avons choisi React.js pour le front-end et Express.js pour le back-end, des choix guidés par les performances et la scalabilité qu'ils offrent. L'hébergement sur Render a été sélectionné pour sa fiabilité et sa facilité d'intégration avec nos piles technologiques.

La gestion de version via Git, avec des revues de code systématiques et des pull-requests, a renforcé l'intégrité et la qualité de notre code. Cela a également permis une documentation complète qui sera cruciale pour les futures mises à jour et la maintenance de l'application.

Respect des Normes et Réglementations

Le projet Deadline Drive adhère aux normes et réglementations en vigueur. Les maquettes et interfaces respectent les recommandations de sécurité des applications web ainsi que les exigences légales, notamment en matière d'accessibilité. Le back-end est conçu pour une gestion sécurisée des données, avec une base de données relationnelle qui garantit l'intégrité et la confidentialité des informations. En outre, nous nous conformons pleinement aux réglementations RGPD, incluant les dispositions sur la protection des données personnelles et la transparence, comme en témoignent notre page de mentions légales et notre politique de confidentialité.

Documentation et Déploiement

La documentation technique, rédigée en anglais et en français, couvre tous les aspects de l'environnement de développement et de déploiement, conformément aux exigences du CECRL niveau B1 pour l'anglais. La procédure de déploiement est bien documentée, avec des scripts prêts à l'emploi qui facilitent la mise en production sécurisée de l'application.

Conclusion

En résumé, le projet Deadline Drive représente une application méthodique des principes agiles pour le développement d'une solution web dynamique et sécurisée. Avec une planification détaillée et une exécution rigoureuse, nous avons réussi à créer une application qui non seulement satisfait les besoins actuels, mais est également prête à s'adapter aux exigences futures.



c. Spécifications fonctionnelles

Codes couleurs utilisés sur Trello pour identifier l'interface du site à laquelle se réfère la user story.

Notification ■ Compte User ■
Citations ■ A propos ■





Gestion des Branches et Développement de Fonctionnalités

Branche Principale : **main**

La branche **main** contient la version la plus stable de l'application, prête pour la production. Elle sert de référence finale pour les déploiements après validation complète des nouvelles fonctionnalités et des correctifs.

Branche de Développement : **dev**

La branche **dev** a fonctionné comme une branche d'intégration pour toutes les fonctionnalités et mises à jour. Chaque branche de fonctionnalité a été fusionnée dans **dev** à la conclusion de son développement. Ceci a permis de réaliser des tests intégrés et de s'assurer que toutes les nouvelles fonctionnalités fonctionnent ensemble avant de fusionner **dev** dans **main** pour le déploiement.

Branche **feat/about**

Développement de la partie front-end de la page "À propos", enrichissant l'expérience utilisateur par des informations contextuelles sur l'application ou l'entreprise.

Branche **feat/comments**

Implémentation du système de commentaires dans une modal sur la page des citations, offrant un CRUD complet en suivant le modèle MVC.

Branche **feat/notifications**

Développement des notifications qui apparaissent dans une modal activée via l'icône de cloche, incluant un backend MVC pour un CRUD complet et un cron job quotidien pour l'envoi de notifications

Branches

[New branch](#)

Overview **Yours** Active Stale All

Branch	Updated	Check status	Behind	Ahead	Pull request
main  	 4 days ago		0	1	 ...
feat/notifications 	 4 days ago		1	0	 #52  ...
feat/comments 	 2 weeks ago		89	0	 #33  ...
feat/passwordrecovery 	 3 weeks ago		122	0	 #28  ...
feat/About 	 3 weeks ago		133	0	 #23  ...
feat/signup_component 	 last month		184	1	 ...
feat/jwt_token 	 last month		191	0	 #14  ...
feat/initialize_frontend 	 last month		206	0	 #10  ...
Branch	Updated	Check status	Behind	Ahead	Pull request
dev  	 4 days ago			Default	 #53  ...



Conclusion

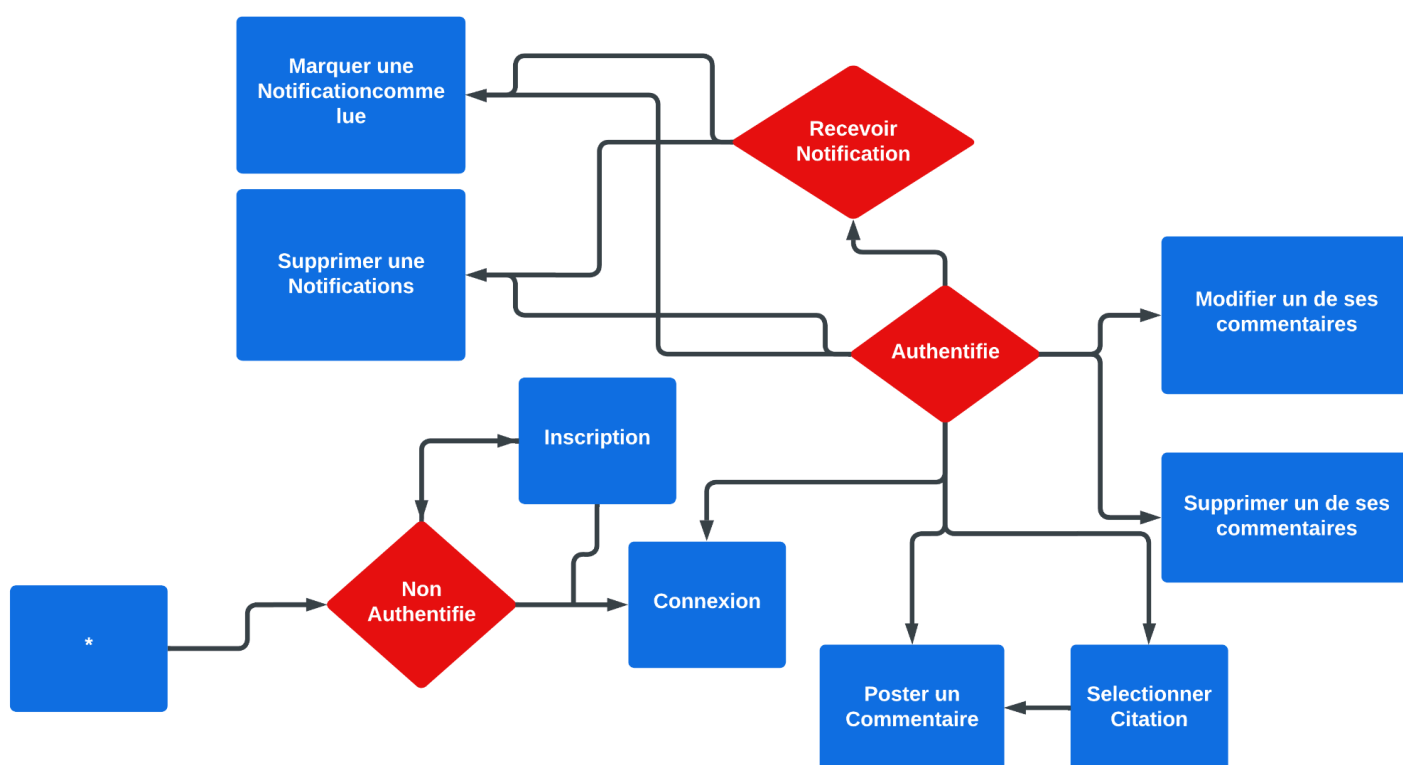
L'utilisation de la branche **dev** comme étape intermédiaire avant la fusion dans **main** a renforcé le processus de développement, en permettant une intégration et des tests plus rigoureux de toutes les fonctionnalités. Cette méthode a non seulement facilité la gestion des mises à jour et des corrections, mais a aussi assuré que la branche **main** demeure stable et prête pour avoir une version en production stable.

Analyse

a. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation est crucial pour planifier notre application web. Il montre les interactions entre les utilisateurs (authentifiés et non authentifiés) et le système, et définit les exigences fonctionnelles clés comme la connexion, l'inscription, la gestion des commentaires, et les notifications.

Créé avec une approche centrée sur l'utilisateur, ce diagramme détaille les étapes spécifiques pour chaque interaction, fournissant une vue d'ensemble claire des fonctionnalités. Il facilite la compréhension pour les développeurs et parties prenantes, en assurant que les fonctionnalités répondent aux attentes des utilisateurs.



b. Wireframes et maquette

Dans le cadre du développement de notre application web, nous avons opté pour une approche structurée pour la conception visuelle de l'interface utilisateur. En utilisant Figma, un outil de design flexible, nous avons d'abord développé des wireframes concernant chaque page du site, autant pour les versions desktop que mobile. Ces wireframes, ont servi de fondation afin de réfléchir sur l'agencement et l'interaction des éléments UI (User Interface), garantissant que chaque composant soit optimisé en vue d'une expérience utilisateur agréable.



Les pages que j'ai spécifiquement traitées dans cette phase initiale comprenaient la Connexion, l'Inscription, la NavBar, le Footer, ainsi que la page "À propos" et la page de récupération de mot de passe. Chacune de ces pages a été esquissée pour assurer la cohérence et l'efficacité de navigation.

Après validation des wireframes, nous sommes passés à la phase de maquettage, toujours en utilisant Figma, pour élaborer des représentations plus détaillées et stylisées de ces pages. Ce processus inclut également la réalisation des maquettes en formats desktop et mobile, en veillant à ce que la transition entre ces deux modes reste fluide et intuitive.

Une attention particulière a été apportée au prototypage dynamique de certains composants interactifs, notamment le FAQ dans la page "À propos". Ce prototypage a permis de simuler le comportement réel des interactions utilisateur, offrant ainsi une anticipation précieuse des réactions et usages réels et ce, en amont du développement technique.

Cette démarche méthodique, de l'esquisse initiale à la maquette finale, a non seulement facilité le développement futur mais a également permis d'aligner précisément nos objectifs de conception avec la charte graphique et les exigences du projet.

Charte Graphique

Palette de couleurs



Police pour titre

Laila

Whereas disregard and contempt for human rights
have resulted

Imprima

Logos



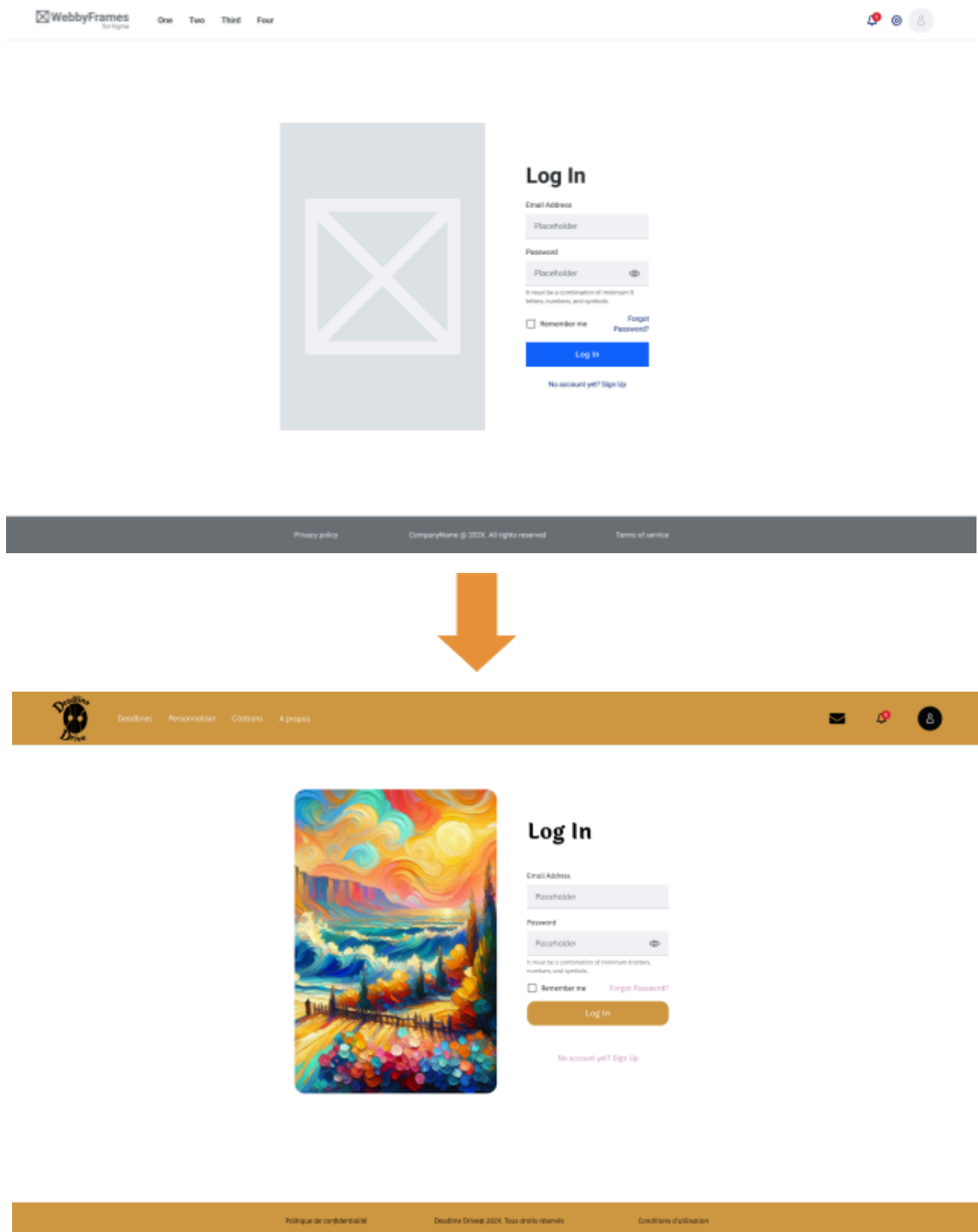
Whereas disregard and contempt for human rights have
resulted

Police pour texte



Evolution du wireframe vers la maquette (Format Bureau)

→ Retrouvez toutes mes maquettes dans les annexes page 59 à 65





Evolution du wireframe vers la maquette (Format Mobile)



NOTRE HISTOIRE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Bibendum amet at molestie mattis.

Rhoncus morbi et augue nec, in id ullamcorper at sit. Condimentum sit nunc in eros scelerisque sed. Commodo in viverra nunc, ullamcorper ut. Non, amet, aliquet scelerisque nullam sagittis, pulvinar. Fermentum scelerisque sit consectetur hac mi. Moris leo eleifend ultricies purus iaculis.

NOTRE EQUIPE



NOS CATEGORIES

Bibendum amet at molestie mattis.

Rhoncus morbi et augue nec, in id ullamcorper at sit. Condimentum sit nunc in eros scelerisque sed. Commodo in viverra nunc, ullamcorper ut. Non, amet, aliquet scelerisque nullam sagittis, pulvinar. Fermentum scelerisque sit consectetur hac mi. Moris leo eleifend ultricies purus iaculis.



NOTRE EQUIPE

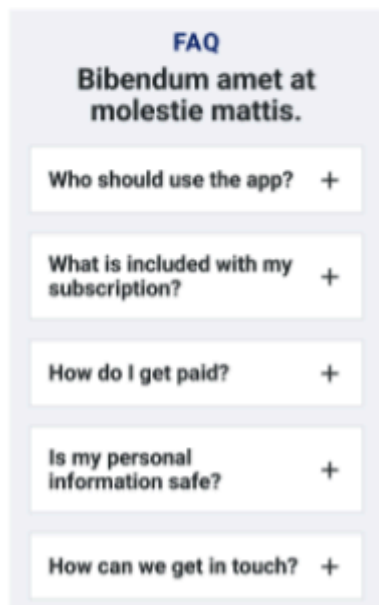


NOTRE HISTOIRE

Imaginez quatre étudiants en développement web, entourés de livres, de café et de code, se battant contre la procrastination comme des chevaliers en quête de motivation. Un jour, alors qu'ils étaient sur le point de succomber à la tentation de procrastiner, l'un d'eux s'écria : "Pourquoi ne pas créer notre propre antidote à la procrastination ?". Et ainsi naquit l'idée de DeadlineDrive - un projet né de leur désir de transformer leur lutte contre la procrastination en une aventure motivante et amusante pour tous. Avec beaucoup de café et de détermination, ils ont conçu une plateforme pour aider les autres à rester motivés, à atteindre leurs objectifs et à célébrer chaque petite victoire comme s'ils étaient en train de conquérir le Saint Graal du succès. Et voilà comment ces quatre étudiants ont transformé leur combat contre la procrastination en une quête épique pour la motivation.

Auteur, Manga, Série, Film, Animé, Histoire, Poésie, Personnalité ou encore Sport

Découvrez l'inspiration à travers une multitude de perspectives avec notre fonctionnalité de citations provenant de différentes catégories. Que vous soyez passionné par les grands écrivains, les mondes fantastiques des mangas et des animés, les intrigues captivantes des séries et des films, les leçons de vie tirées de l'histoire, les vers envoûtants de la poésie, les enseignements des personnalités influentes ou les exploits inspirants du monde du sport, vous trouverez une source d'inspiration adaptée à vos goûts et à vos intérêts pour nourrir votre esprit et stimuler votre motivation chaque jour.



CompanyName © 202X. All rights reserved.

[Privacy policy](#) [Terms of service](#)

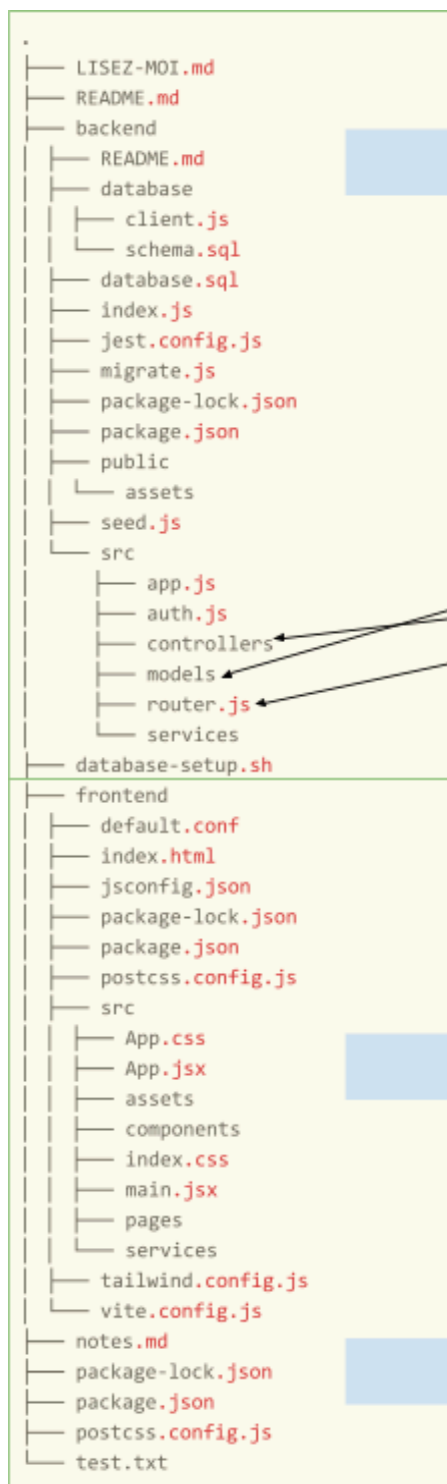


Deadline Drive© 202X. Tous droits réservés

[Politique de confidentialité](#) [Conditions Générales d'utilisation](#)



c. Arborescence



Backend

La gestion des données de l'application se trouve dans le dossier **backend**. Elle contient les fichiers de configuration, les scripts de migration de la base de données, les fichiers de configuration de test, et les scripts de semence de données. Le dossier **database** contient les fichiers relatifs à la configuration et au schéma de la base de données. Le dossier **src** contient les fichiers principaux de l'application backend.

MVC

La structure MVC (Model-View-Controller) est organisée comme suit dans le dossier **src** du backend :

- **Models** : Définit les structures de données et les interactions avec la base de données.
- **Controllers** : Contient la logique d'application, gérant les requêtes et les réponses.
- **Views** : Non spécifié explicitement, car cette architecture est plus orientée API, les vues pourraient être les réponses JSON envoyées au frontend.

Frontend

Tous les éléments d'affichage sont situés dans le dossier **frontend**. Elle contient les configurations et les fichiers nécessaires pour exécuter l'application client, principalement construite avec React.

- **App.jsx** : Point d'entrée principal de l'application React. Il configure les routes en utilisant React Router.
- **Components** : Contient des composants réutilisables qui forment les différentes parties de l'interface utilisateur.
- **Pages** : Contient les composants de page, instanciés par React Router via le fichier **App.jsx** pour chaque route définie.

Node Modules

Les dossiers **node_modules** sont présents à la fois dans les parties backend et frontend du projet. Ils contiennent les bibliothèques et dépendances nécessaires au fonctionnement de chaque partie de l'application. Le backend utilise des modules Node.js pour le serveur et la base de données, tandis que le frontend utilise des modules pour React, le routage, et le styling, entre autres.



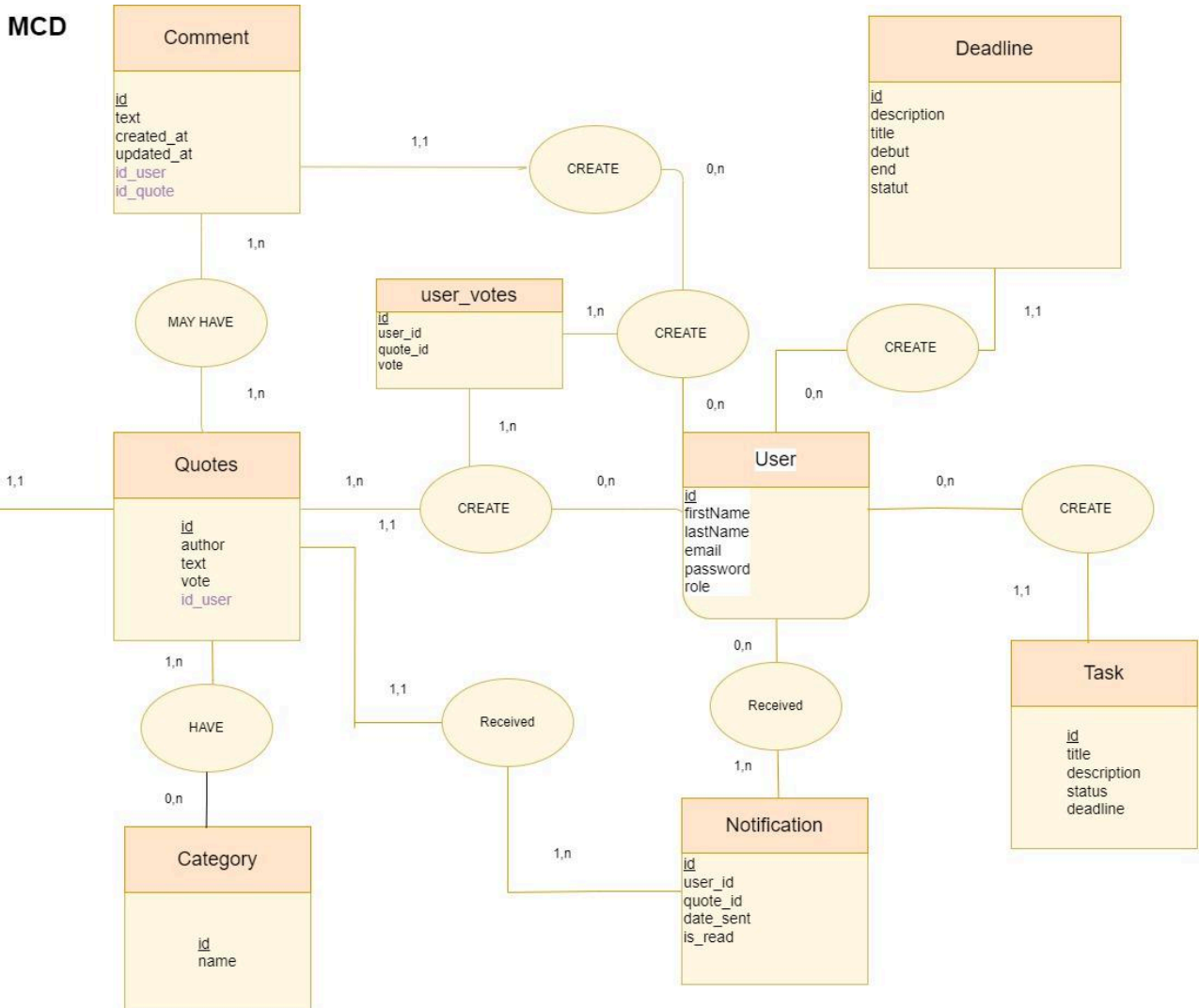
Base de données

Introduction à la Conception de Base de Données avec Merise

La méthode Merise est largement utilisée pour la conception de systèmes d'information, notamment les bases de données. Cette approche structurée permet de modéliser un système d'information à différents niveaux d'abstraction, facilitant ainsi le développement, l'implémentation et la maintenance. La méthode Merise se décompose en plusieurs étapes et modèles, dont les principaux sont le Modèle Conceptuel de Données (MCD), le Modèle Logique de Données (MLD), et le Modèle Physique de Données (MPD).

a. Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données représente de façon abstraite la structure de la base de données sans se soucier des contraintes techniques. Il se concentre sur les entités, leurs attributs et les relations entre ces entités. L'utilisation du MCD permet de décrire le domaine d'information de façon claire et organisée.





Analyse du MCD

En examinant le MCD fourni, nous identifions plusieurs entités clés qui structurent l'application de gestion de commentaires et de citations :

1. **Comment** : Représente les commentaires laissés par les utilisateurs. Chaque commentaire est lié à un utilisateur et à une citation.
2. **User** : Détaille les informations sur les utilisateurs de l'application, incluant leur nom, prénom, email et mot de passe.
3. **Quotes** : Contient des citations qui peuvent être commentées, votées ou être mises en favoris par les utilisateurs. Chaque citation est également liée à une catégorie spécifique.
4. **Category** : Catégorise les citations.
5. **User_Votes** : Gère les votes des utilisateurs sur les citations pour exprimer leur appréciation.
6. **Favorites** : Permet aux utilisateurs de marquer des citations comme favorites.
7. **Notification** : Gère les notifications envoyées aux utilisateurs, par exemple, chaque jour il est envoyé une notification aux utilisateurs inscrit avec une citation prise au hasard dans la base de données.
8. **Task** : Représente des tâches ou des activités avec des échéances spécifiques, pouvant être liées à des utilisateurs ou des citations.
9. **Deadline** : Représente des deadlines avec des échéances spécifiques, pouvant être liées à des utilisateurs ou des citations.

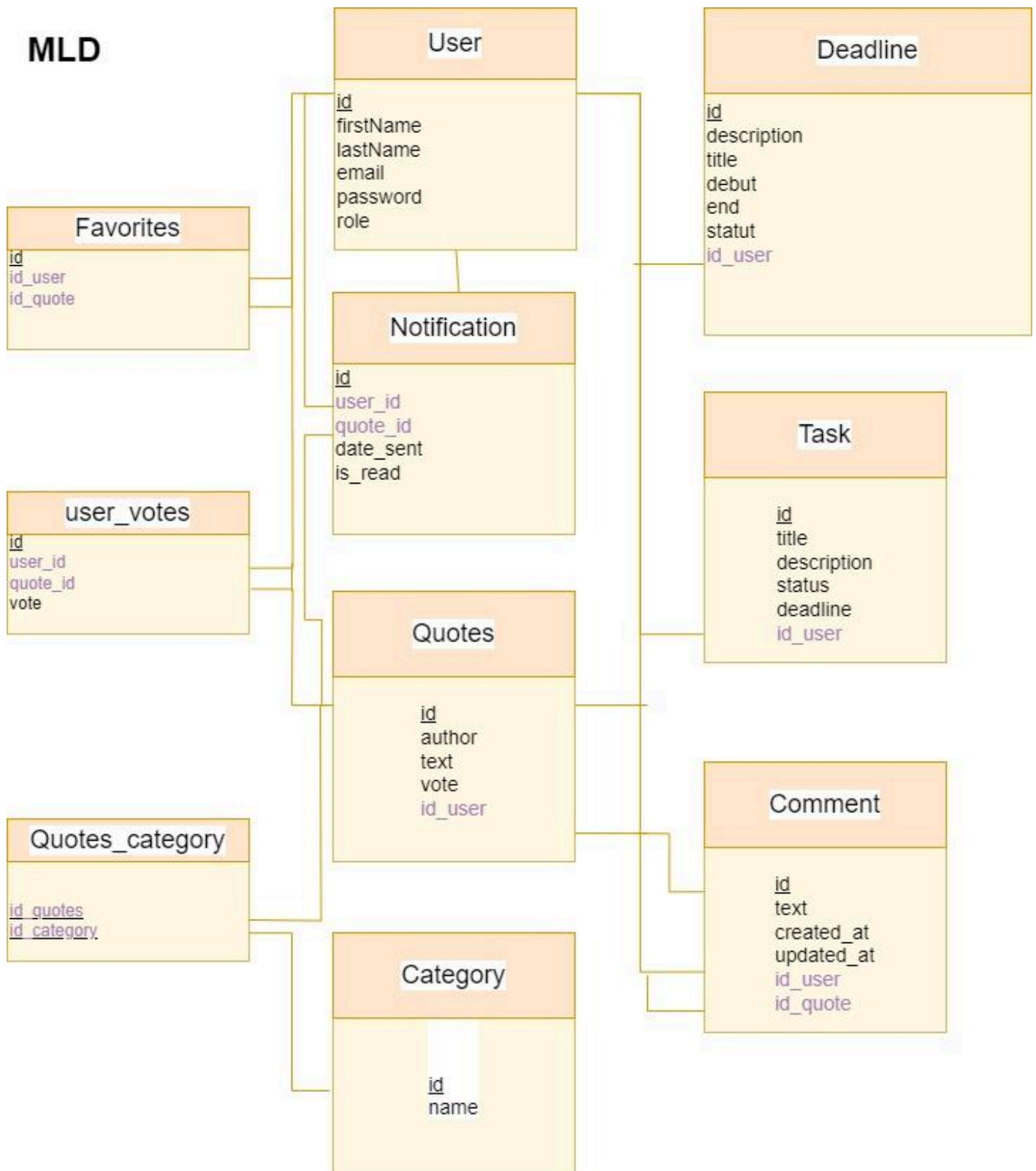
Ces entités sont interconnectées par des relations qui expriment les interactions entre elles, telles que les relations "MAY HAVE" entre les commentaires et les citations, ou les relations "HAVE" qui définissent la possession ou l'affiliation, comme un utilisateur ayant des favoris ou recevant des notifications.

b. Modèle Logique de Données (MLD)

Le Modèle Logique de Données (MLD) transforme le Modèle Conceptuel de Données (MCD) en une structure adaptée à la mise en œuvre sur une base de données relationnelle. Cela implique de définir des tables, des clés primaires pour identifier de manière unique les enregistrements dans chaque table, des clés étrangères pour établir des relations entre les tables, ainsi que d'autres contraintes et indices pour optimiser les performances et l'intégrité des données.



MLD





Analyse du MLD

Le MLD illustre la transition des entités conceptuelles vers les structures de base de données relationnelle spécifiques. Voici les détails des tables et relations clés :

- **User** : Contient les informations détaillées des utilisateurs, incluant leurs identifiants, noms, prénoms, emails, mots de passe, et autres détails personnels. Cette table est centrale car elle se connecte à plusieurs autres tables comme **Comment**, **Favorites**, et **Notification**.
- **Comment** : Enregistre les commentaires que les utilisateurs publient sur les citations. Chaque commentaire est lié à un utilisateur et à une citation via **id_user** et **id_quote**.
- **Quotes** : Stocke les citations, chaque citation à un auteur, un type, un texte et peut être liée à un utilisateur ainsi qu'à une catégorie via une table de liaison **Quotes_category**.
- **Category** : Détient les différentes catégories pouvant être attribuées aux citations pour faciliter leur classification et recherche.
- **Quotes_category** : Une table de liaison qui gère la relation plusieurs-à-plusieurs entre les citations et les catégories, permettant à une citation d'appartenir à plusieurs catégories.
- **Favorites** : Permet aux utilisateurs de marquer certaines citations comme favorites, cette table enregistre les relations entre les utilisateurs et les citations qu'ils ont favorisées.
- **Notification** : Chaque jour, tous les utilisateurs reçoivent une citation au hasard prise dans la base de données.
- **Task** : Cette table stocke des tâches attribuées aux utilisateurs, avec des informations telles que le titre, la description, et une échéance.
- **Deadline** : Concerne spécifiquement les informations sur les échéances, qui peuvent être liées à différentes tâches.
- **User_votes** : Enregistre les votes des utilisateurs sur les citations. Cette fonctionnalité permet aux utilisateurs d'exprimer leur appréciation des citations.

Importance du MLD

Le MLD est crucial car il :

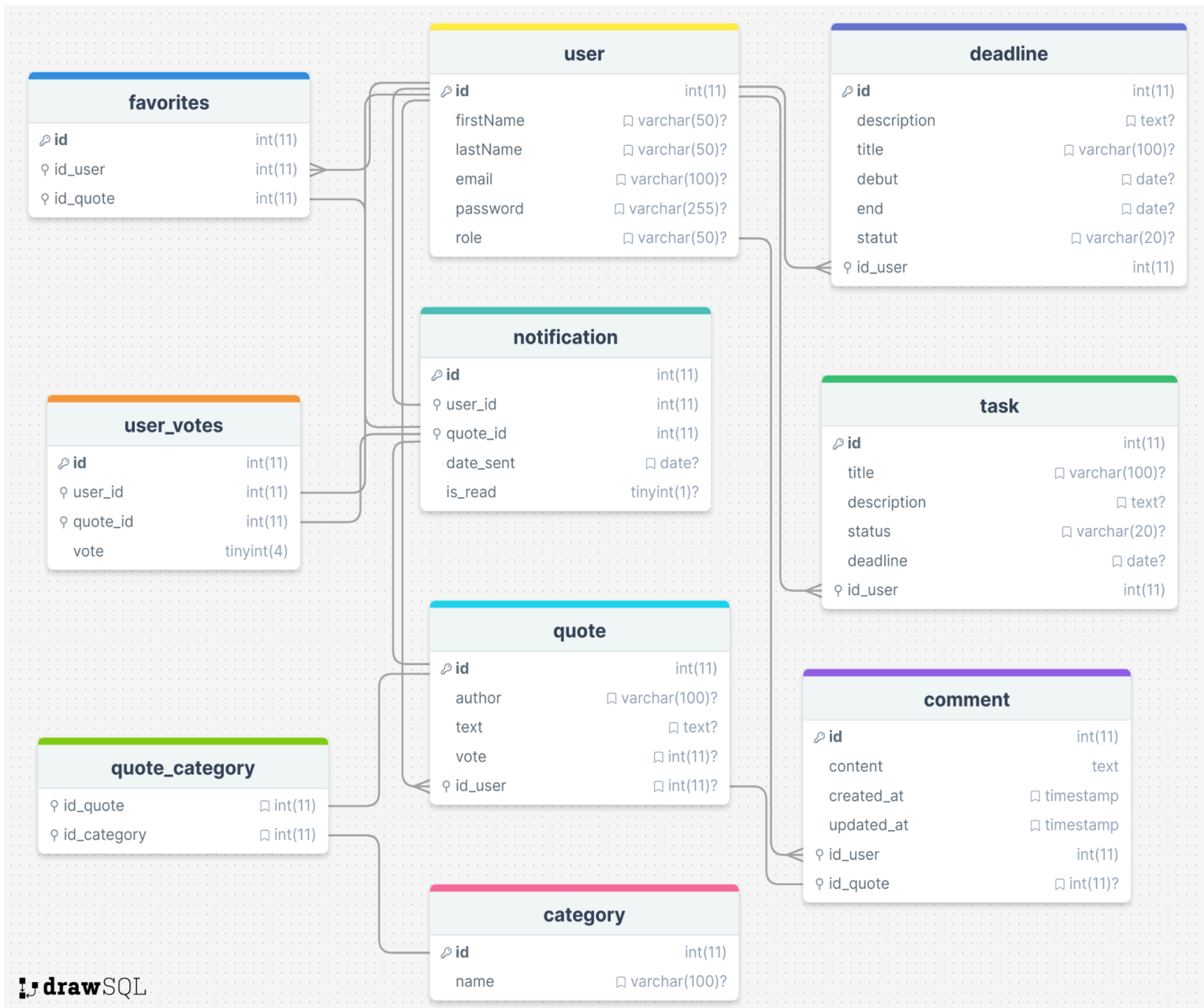
- **Assure l'intégrité des données** : Les clés étrangères et les contraintes de table aident à maintenir la précision et la consistance des données stockées.
- **Optimise les performances** : Les indices et les clés primaires permettent des recherches plus rapides et des opérations de jointure efficaces.
- **Facilite la maintenance** : Une structure logique claire aide à la maintenance et à l'évolution de la base de données.

Dans le passage au Modèle Physique de Données, nous traduisons ce schéma logique en instructions SQL spécifiques pour créer la base de données, en utilisant le Langage de Définition de Données pour définir physiquement la structure et les contraintes dans le système de gestion de base de données.



c. Modèle Physique de Données (MPD)

Le Modèle Physique de Données (MPD) est la représentation détaillée du modèle de données conçu pour être implémenté au sein d'un système de gestion de base de données. Ce modèle inclut les tables spécifiques, les types de données pour chaque colonne, les clés primaires, les clés étrangères, et d'autres éléments tels que les contraintes de non-nullité et les valeurs par défaut. Le MPD ci-dessous illustre une approche pragmatique où certaines colonnes peuvent être laissées avec la possibilité d'être nulle afin de permettre une flexibilité initiale et faciliter le débogage interne.





Description Détaillée du MPD

Table user	Colonnes: id, firstName, lastName, email, password, role. Contraintes: id est la clé primaire. Remarques: Les champs comme email et password sont essentiels et pourraient être rendus NOT NULL pour renforcer l'intégrité des données.
Table comment	Colonnes: id, content, created_at, updated_at, id_user, id_quote. Contraintes: id est la clé primaire; id_user et id_quote sont des clés étrangères reliant aux tables user et quote. Remarques: id_quote peut être NULL, ce qui permet d'ajouter des commentaires qui ne sont pas directement liés à une citation.
Table quote	Colonnes: id, author, text, vote, id_user. Contraintes: id est la clé primaire; id_user est une clé étrangère qui se réfère à user. Remarques: Le système permet de voter sur les citations, ce qui est indiqué par le champ vote.
Table category et quote_category	Gestion des Catégories: Les citations appartiennent à une catégorie, ce qui est géré via une table d'association quote_category.
Table task et deadline	Gestion des Tâches et Échéances: Les tâches sont associées à des utilisateurs et peuvent avoir des échéances. La distinction entre task et deadline permet une gestion plus flexible des délais.
Table favorites et user_votes	Favoris et Votes: Les utilisateurs peuvent marquer des citations comme favorites et voter pour elles, ce qui est géré via les tables favorites et user_votes.

Flexibilité et Sécurité des Données

- **Champs NULL:** Laisser des champs comme NULL dans la phase initiale peut aider dans le débogage interne et offre une certaine flexibilité lors du développement. Cela permet aux développeurs de tester différentes configurations sans contraintes strictes.
- **Sécurisation Future :** Dans des versions ultérieures, il serait prudent de rendre certains champs NOT NULL pour renforcer l'intégrité des données. Cela inclut des champs critiques comme email, password, firstName, et lastName dans la table user.



Conclusion

Le MPD fourni démontre un équilibre entre flexibilité initiale et structuration des données pour un développement efficace. Il est essentiel de continuer à évaluer les besoins de l'application et de modifier la base de données afin d'améliorer la sécurité et l'efficacité au fur et à mesure que l'application évolue et que plus d'utilisateurs interagissent avec elle. En pratique, la collaboration entre les équipes de développement front-end et back-end est cruciale pour assurer que les contraintes appliquées dans la base de données correspondent aux validations réalisées côté client.

d. Langage de Définition de Données (LDD)

Le Langage de Définition de Données, ou Data Definition Language (DDL) joue un rôle crucial dans la création, la modification et la gestion des structures de bases de données. Ce langage permet aux développeurs de spécifier divers schémas de données, de définir des relations entre les données, et de gérer les bases de données au niveau du système de gestion de base de données (SGBD). Les commandes les plus courantes du DDL incluent CREATE, ALTER, et DROP.

Commandes DDL Clés

CREATE DATABASE	<pre>CREATE DATABASE deadlinedrive;</pre>
CREATE TABLE Utilisée pour créer une nouvelle table dans la base de données. Exemple de syntaxe pour créer une table selon le modèle de notre application.	<pre>CREATE TABLE User (id INT AUTO_INCREMENT PRIMARY KEY, firstName VARCHAR(50), lastName VARCHAR(50), email VARCHAR(100) NOT NULL, password VARCHAR(100) NOT NULL, role VARCHAR(50));</pre>
ALTER TABLE Permet de modifier la structure d'une table existante, comme ajouter ou supprimer des colonnes, changer le type de données d'une colonne, ou ajouter des contraintes.	Exemple d'ajout d'une colonne à une table existante : <pre>ALTER TABLE User ADD COLUMN birthday DATE;</pre> Exemple de modification du type de données pour une colonne existante: <pre>ALTER TABLE User MODIFY COLUMN email VARCHAR(30);</pre>
DROP TABLE Utilisée pour supprimer complètement une table existante dans la base de données, ainsi que toutes ses données, sa structure et ses contraintes associées.	Cette commande supprime la table nommée "clients" de la base de données. <pre>DROP TABLE User;</pre>



Utilisation des Commandes DDL pour la Structuration de la Base de Données

Dans la pratique, les commandes DDL sont utilisées non seulement pour initier la structure de la base de données, mais également pour maintenir et adapter cette structure au fil du temps en fonction des besoins évolutifs de l'application. Cela comprend l'ajout de nouvelles fonctionnalités, la correction de modèles de données inefficaces ainsi que la réponse aux exigences réglementaires et de sécurité.

Exemple complet d'utilisation du DDL pour la Base de Données de l'Application

Pour illustrer la mise en place du MPD de notre application, voici quelques commandes DDL supplémentaires qui définissent les autres tables. Ces commandes créent les tables [User](#), [Category](#), [Quote](#), [Comment](#), et [Notification](#) en spécifiant les colonnes nécessaires, les types de données, et les contraintes, telles que les clés primaires et étrangères.

Les tables sont conçues pour assurer l'intégrité référentielle et faciliter les relations entre les utilisateurs, les citations, les commentaires et les notifications.

Exemple :

```
CREATE TABLE User (
    id INT AUTO_INCREMENT PRIMARY KEY,
    firstName VARCHAR(50),
    lastName VARCHAR(50),
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL,
    role VARCHAR(50),
);
CREATE TABLE Category (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL
);
CREATE TABLE Comment (
    id INT AUTO_INCREMENT PRIMARY KEY,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    id_user INT,
    id_quote INT,
    FOREIGN KEY (id_user) REFERENCES User(id),
    FOREIGN KEY (id_quote) REFERENCES Quote(id)
);
CREATE TABLE Notification (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    quote_id INT,
    date_sent DATE,
    is_read TINYINT(1) DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES User(id),
    FOREIGN KEY (quote_id) REFERENCES Quote(id)
);
```




Développement

a. Structure du projet

Une documentation en anglais est disponible dans le fichier [README.md](#). Cette documentation est rédigée pour vous guider à travers les divers aspects de notre projet, abordant les défis, les fonctionnalités, et les spécifications techniques. Voici un aperçu de ce que vous trouverez dans ce document :

- **Aperçu du Projet** : Introduction au projet, exposant ses objectifs et les problèmes qu'il vise à résoudre.
- **Fonctionnalités** : Descriptions détaillées des fonctionnalités qui améliorent l'expérience utilisateur et les capacités du système.
- **Configuration de l'environnement de travail** : Instructions pour configurer l'environnement nécessaire pour démarrer et exécuter le projet.
- **Processus d'Installation** : Instructions étape par étape pour vous aider à installer et configurer le projet facilement.
- **Cas d'utilisation** : Scénarios et cas d'utilisation pour montrer comment l'application peut être utilisée efficacement.
- **Spécifications Techniques** : Un examen approfondi de l'architecture technique, incluant les cadres, les bibliothèques, et les langages de programmation utilisés.
- **Public cible** : Informations sur le public ciblé et comment les différents utilisateurs peuvent en bénéficier.

Ce [README](#) sert de guide pour naviguer et comprendre notre projet, garantissant que tous les participants, des développeurs aux utilisateurs finaux, disposent des ressources nécessaires pour une interaction réussie avec notre application.

```
Extrait de la documentation

# Setup & Use
## Project Initialization
Clone the repository and navigate into it:

```bash
git clone <repository_url>
cd <repository_name>
```

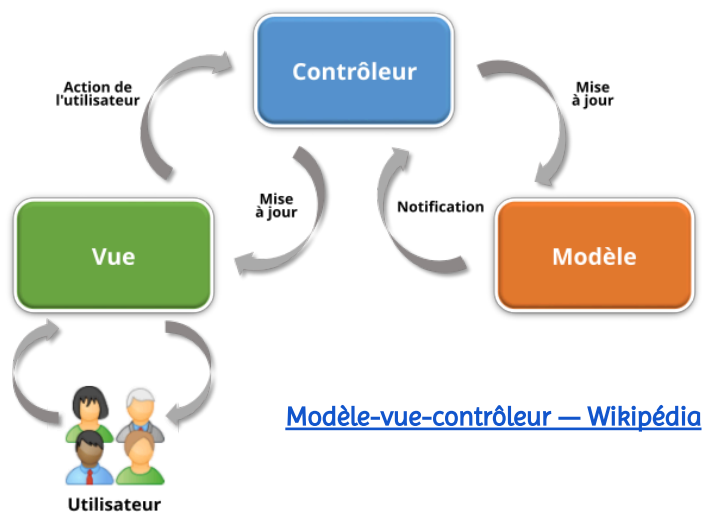
Install dependencies using npm:

```bash
npm install
Backend Server Setup:
```

The backend is hosted on port 5000, and the frontend on port 5173.
Configure the backend .env file with your database credentials for AlwaysData.
plaintext
```

Documentation complète:

<https://github.com/KD63799/deadlinedrive/blob/dev/README.md>



Dans ce projet fullstack, le backend est développé en Node.js tandis que le front-end utilise React.js. Ensemble, sur notre serveur back-end nous mettons en œuvre le design pattern de conception MVC (Model-View-Controller), un schéma architectural qui aide à structurer le code de manière efficace en séparant les responsabilités.

Fonctionnement du Modèle MVC

Le modèle MVC divise l'application en trois composants principaux, chacun spécialisé dans une fonction spécifique :

Le Modèle : Géré par JavaScript dans le backend, le modèle gère les interactions avec la base de données. Il récupère, insère, met à jour et supprime les données nécessaires selon les demandes qu'il reçoit du contrôleur.

La Vue : La partie vue n'était pas présente dans notre logique backend. React.js assume en quelque sorte le rôle de la "vue" dans notre architecture. Il est chargé de présenter les données à l'utilisateur, en rendant l'interface utilisateur dynamique. React reçoit les données sous forme de JSON du backend et réagit aux interactions de l'utilisateur afin de créer une expérience fluide et réactive.

Le Contrôleur : Également géré par JavaScript dans le backend, le contrôleur traite la logique métier de l'application. Il reçoit les entrées des utilisateurs via l'API rest, les valide conformément aux règles métier puis interagit avec le modèle pour effectuer des opérations sur les données. Les résultats sont ensuite formatés, souvent en JSON, puis renvoyés au client, puis utilisés par React pour l'affichage.

Cette structuration en MVC permet une gestion claire du code : le backend gère la logique et les données, tandis que React dans le front-end s'occupe de la présentation, assurant ainsi que chaque partie de l'application peut évoluer de manière indépendante tout en maintenant une intégration cohérente. Dans la structure du projet, l'implémentation des opérations CRUD (Create, Read, Update, Delete) est un élément central qui assure la gestion efficace des données à travers différentes parties de l'application. Ces opérations sont fondamentales pour interagir avec la base de données et sont utilisées pour manipuler les données de manière cohérente et sécurisée.



Implémentation du CRUD

Chaque manager dans l'architecture du projet, tel que **NotificationManager** et **CommentManager**, intègre des méthodes CRUD qui permettent de créer, lire, mettre à jour, et supprimer des données spécifiques. Ces méthodes garantissent que les interactions avec la base de données sont réalisées via une interface bien définie, réduisant les risques d'erreurs et facilitant ainsi la maintenance du code.

Exemples de méthodes CRUD :

| | |
|---|--|
| Create : Les méthodes create dans les classes de gestion telles que NotificationManager et CommentManager permettent d'insérer de nouvelles entrées dans la base de données. Par exemple, la méthode create dans NotificationManager insère une nouvelle notification en utilisant des paramètres tels que userId , quoteId , ainsi que la date actuelle. | <pre>create(userId, quoteId) { const dateSent = new Date().toISOString().slice(0, 10); return this.database.query(`INSERT INTO notification (user_id, quote_id,     date_sent, is_read) VALUES (?, ?, ?, FALSE)`, [userId, quoteId, dateSent]); }</pre> |
| Read : Les méthodes telles que findAll ou find récupèrent des données de la base de données. Ces méthodes sont utilisées pour extraire soit toutes les entrées d'une table, soit une entrée spécifique par son identifiant. | <pre>const browse = (req, res) => { models.user.findAll() .then(([rows]) => { res.send(rows); }) .catch((err) => { console.error(err); res.sendStatus(500);});}</pre> |
| Update : Les méthodes update et markAsRead dans NotificationManager modifient les données existantes. Par exemple, markAsRead met à jour le statut de lecture d'une notification pour indiquer qu'elle a été lue. | <pre>markAsRead(notificationId) { return this.database.query(`UPDATE notification SET is_read = TRUE WHERE     id = ?`, [notificationId]);}</pre> |
| Delete : Les méthodes delete permettent de supprimer des entrées de la base de données. Cette opération est centrale pour gérer le cycle de vie des données et assurer que les données obsolètes ou inutiles ne saturent pas la base de données. | <pre>const destroy = (req, res) => { models.user.delete(req.params.id) .then(([result]) => { if (result.affectedRows === 0) {res.sendStatus(404); } else {res.sendStatus(204); }}) .catch((err) => { console.error(err); res.sendStatus(500); });});}</pre> |



Avantages du CRUD

L'utilisation systématique des opérations CRUD dans le projet offre plusieurs avantages:

1. **Uniformité** : L'interface uniforme pour toutes les opérations de base de données simplifie le développement et la maintenance en offrant une méthode cohérente pour toutes les interactions avec la base de données.
2. **Sécurité** : En centralisant la gestion des données dans des managers spécifiques, on réduit le risque d'injection SQL ainsi que d'autres vulnérabilités en validant et en contrôlant toutes les entrées et sorties de la base de données.
3. **Maintenabilité** : La modification des opérations de base de données est simplifiée, car elles sont centralisées dans des classes spécifiques. Cela permet des mises à jour plus rapides et moins sujettes aux erreurs, facilitant l'évolution de l'application.

L'implémentation soignée des opérations CRUD dans ce projet est donc essentielle non seulement pour le fonctionnement quotidien de l'application mais aussi pour sa longévité, sa capacité à évoluer et à s'adapter aux besoins futurs.

Connexion et Inscription des Utilisateurs

La gestion des utilisateurs est un aspect essentielle de l'application, permettant de maintenir un environnement sécurisé et personnalisé pour chaque utilisateur. Les fonctions de connexion et d'inscription sont implémentées en utilisant des méthodes de cryptage avancées et des jetons JWT pour la gestion des sessions.

Inscription : Lors de l'inscription, les informations de l'utilisateur y compris le mot de passe sont recueillies. Le mot de passe est immédiatement haché à l'aide de l'algorithme Argon2, réputé pour sa sécurité et sa résistance aux attaques par force brute. Une fois le mot de passe haché, les informations de l'utilisateur sont stockées dans la base de données. Voici un exemple de la route d'inscription qui utilise la fonction de hachage :

```
router.post("/users", hashPassword, userControllers.add);
```

Dans cette route, `hashPassword` est un middleware qui prend le mot de passe en clair fourni par l'utilisateur, le hache, et remplace le mot de passe en clair dans l'objet de la requête par sa version hachée avant de passer au contrôleur, qui gère l'ajout de l'utilisateur à la base de données.

Connexion : La connexion est gérée par un processus qui vérifie les identifiants fournis par l'utilisateur contre ceux stockés dans la base de données. Si les identifiants sont identiques, un jeton JWT est généré et envoyé au client. Ce jeton est utilisé pour les requêtes subséquentes à l'API rest pour authentifier l'utilisateur. Le processus utilise le middleware `verifyPassword`, qui s'assure que le mot de passe fourni correspond au hachage stocké dans la base de données.

```
router.post("/users/login",  
userControllers.getUserByEmailAndPasswordAndPassToNext, verifyPassword);
```

Dans cet exemple, `getUserByEmailAndPasswordAndPassToNext` est un middleware qui récupère l'utilisateur par son email et le passe à `verifyPassword` pour validation. Si la validation réussit, `verifyPassword` génère un JWT et transmet au client qui peut l'utiliser pour maintenir l'état de connexion.



Utilisation des JWT

Les JWT sont des jetons compacts, sûrs et autovérifiables, idéaux pour l'authentification dans les applications web modernes. Chaque JWT contient un ensemble de revendications encodées sous forme JSON, qui peuvent inclure l'identifiant de l'utilisateur et d'autres métadonnées nécessaires. Après la connexion, chaque requête du client inclura le JWT, habituellement dans l'en-tête **Authorization**, permettant au serveur de vérifier rapidement l'identité de l'utilisateur et de fournir une réponse appropriée.

Avantages de l'approche JWT

1. **Sécurité** : Les JWT sont signés numériquement, ce qui garantit que le contenu n'a pas été modifié.
2. **Stateless** : Les JWT permettent une authentification sans état. Le serveur n'a pas besoin de garder la session utilisateur; chaque requête est autonome avec le jeton d'authentification.
3. **Performance** : L'authentification "sans état" réduit la nécessité de requêtes de base de données fréquentes pour récupérer les détails de l'utilisateur, ce qui peut améliorer la performance de l'application.

Ces mécanismes d'inscription, de connexion, et d'authentification JWT constituent le socle de la sécurité et de la gestion des utilisateurs dans le projet, assurant que les opérations sensibles sont protégées et que les interactions utilisateurs sont fluides et sécurisées.

Dans la structure du projet, le frontend joue un rôle crucial en offrant une interface utilisateur riche et interactive. Utilisant React.js, une bibliothèque JavaScript populaire pour la construction d'interfaces utilisateur, le projet tire parti de ses capacités de composants dynamiques et réactifs pour offrir une expérience utilisateur fluide et réactive. Les technologies et méthodologies choisies pour le frontend sont essentielles pour assurer non seulement l'efficacité du développement mais aussi une expérience utilisateur optimisée.

Dans la conception du frontend de ce projet, Tailwind CSS a été sélectionné comme cadre CSS principal pour son approche utilitaire et sa flexibilité, facilitant la création d'une interface utilisateur responsive qui adhère également à la charte graphique du projet. Cette intégration de Tailwind CSS souligne l'engagement envers un design esthétique cohérent tout en offrant une expérience utilisateur optimisée sur différents appareils.

Tailwind CSS pour le Design Responsive

Tailwind CSS est particulièrement apprécié pour sa capacité à construire rapidement des designs responsives sans s'éloigner de la charte graphique définie. Contrairement aux frameworks CSS traditionnels qui proposent des composants préconçus, Tailwind permet aux développeurs de créer des designs sur mesure en utilisant des classes utilitaires directement dans le HTML (ou JSX dans le cas de React). Ce système permet une personnalisation complète et un contrôle précis sur le style des applications, facilitant l'adhérence stricte à une charte graphique spécifique.

Avantages de Tailwind CSS dans le Projet

1. **Rapidité de Développement** : Tailwind CSS offre des centaines de classes utilitaires qui permettent aux développeurs de styliser directement dans le balisage, réduisant le besoin de naviguer entre les feuilles de style CSS séparées. Cela accélère considérablement le processus de développement.



2. **Consistance visuelle** : En utilisant Tailwind CSS, le projet bénéficie d'une cohérence visuelle à travers toutes les pages et composants. Les développeurs peuvent réutiliser les mêmes classes utilitaires pour maintenir une apparence uniforme, ce qui est essentiel pour respecter la charte graphique.
3. **Responsive Design** : Tailwind intègre des fonctionnalités de responsive design par défaut. En utilisant des préfixes pour les tailles d'écran dans les classes, il est simple de modifier l'apparence d'un élément à différentes résolutions, assurant que l'application est aussi fonctionnelle et esthétiquement agréable sur mobile que sur desktop.
4. **Maintenance facilitée** : Avec Tailwind, les styles sont généralement plus cohérents et plus faciles à gérer. La mise à jour ou la modification de l'UI peut être réalisée plus rapidement, ce qui réduit le coût et la complexité de la maintenance.

Utilisation de Tailwind CSS dans le Projet

Dans ce projet, Tailwind CSS a été utilisé pour styliser des composants tels que des formulaires d'inscription et de connexion, des barres de navigation, et des boutons. La facilité d'ajouter des marges, des paddings, des tailles de police mais également des couleurs a permis aux développeurs de créer des interfaces conformes aux spécifications de design sans écrire de CSS complexe.

Cette approche maintient non seulement la cohérence avec la charte graphique de l'entreprise mais garantit également que l'application reste accessible et performante sur tous les appareils et résolutions. Tailwind CSS, par son approche modulaire et responsive, est ainsi devenu un outil indispensable dans le toolkit de développement du projet, assurant que l'interface utilisateur reste moderne, réactive et conforme aux attentes des utilisateurs finaux.

b. Front-end

Le composant **NavBar** développé pour ce projet illustre l'utilisation de React.js pour créer une barre de navigation dynamique et responsive qui améliore significativement l'interface utilisateur (UI) et l'expérience utilisateur (UX) du site web. Ce composant est riche en fonctionnalités et optimisé pour la navigation entre différentes parties de l'application tout en offrant un design élégant et accessible.

Caractéristiques Principales de la NavBar

1. Intégration de React Router : La **NavBar** utilise React Router pour la navigation, ce qui permet un routage déclaratif au sein de l'application. Cela inclut l'utilisation de `<Link>` pour des transitions de page sans rechargement, garantissant une expérience utilisateur fluide et rapide.

→ Utilisation du composant `<Link>` pour la navigation sans rechargement :

```
<Link to="/"> <img src={logo} alt="Logo" className="h-16 w-16 mr-2" /> </Link>
```

2. Gestion des États Locaux avec les Hooks : L'état local du composant est géré avec les hooks `useState` de React. Cela inclut les états pour gérer l'ouverture et la fermeture du menu mobile (`isOpen`), la gestion des modales pour les notifications (`isModalOpen`), et le contrôle des menus déroulants (`isDropdownOpen`). Chaque état est modifié par des fonctions spécifiques (`toggleMenu`, `toggleModal`, `handleMouseEnter`, `handleMouseLeave`) démontrant une gestion d'état claire et efficace.



→ Utilisation de `useState` pour gérer les états du menu mobile et de la modal des notifications:

```
const [isOpen, setIsOpen] = useState(false);
const [isModalOpen, setIsModalOpen] = useState(false);
const toggleMenu = () => {
  setIsOpen(!isOpen);
};
const toggleModal = () => {
  setIsModalOpen(!isModalOpen);
};
```

3. Utilisation de Framer Motion pour les Animations : La librairie Framer Motion est utilisée pour animer les éléments de la navbar, comme les liens et le menu mobile. Des variantes sont définies pour chaque animation, comme `linkVariants` pour les liens et `menuVariants` pour le menu mobile, permettant des animations fluides et esthétiques à l'ouverture et à la fermeture de ces éléments.

→ Définition des variantes d'animation pour les liens et le menu mobile :

```
const linkVariants = {
  hover: {
    scale: 1.1,
    transition: { duration: 0.3 },
  },
};
const menuVariants = {
  opened: {
    x: 0,
    opacity: 1,
    display: "block",
    transition: { duration: 0.5 },
  },
  closed: {
    x: "100%",
    opacity: 0,
    transitionEnd: {
      display: "none",
    },
    transition: { duration: 0.5 },
  },
};
```

4. Composants et Iconographie de Hero Icons : Des icônes de la bibliothèque Hero Icons sont intégrées pour enrichir visuellement les liens et les boutons. Ces icônes rendent l'interface plus intuitive et attrayante, facilitant la navigation pour les utilisateurs.

→ Utilisation des icônes Hero Icons dans les boutons interactifs :

```
[
  { icon: <EnvelopeIcon className="h-7 w-7" />, to: '/contact' },
  { icon: <BellIcon className="h-7 w-7" />, onClick: toggleModal },
  { icon: <UserIcon className="h-7 w-7" />, to: '/user-account' }
].map((item, index) => ( item.to ? (
  <Link to={item.to} className="text-custom-black hover:text-gray-200"
  key={index}> {item.icon} </Link> ) : ( <button onClick={item.onClick}
  className="text-custom-black hover:text-gray-200" key={index}>
  {item.icon} </button> ) )))
```



5. Responsive Design : La **NavBar** est conçue pour être responsive, avec des styles adaptés pour les écrans mobiles gérés par des variantes de Framer Motion et des classes CSS de Tailwind. Ceci est particulièrement visible dans la gestion du menu mobile, qui est activé avec un bouton dans les vues de type "mobile".

→ *Adaptation aux écrans mobiles avec un bouton pour afficher ou masquer le menu :*

```
<motion.button      onClick={toggleMenu}      className="text-custom-black
hover:text-gray-200 inline-flex items-center justify-center p-2
rounded-md focus:outline-none">
  {isOpen ? <XMarkIcon className="h-6 w-6" /> : <Bars3Icon
className="h-6 w-6" />}
</motion.button>
```

6. Composant Notifications : Un sous-composant **Notifications** est intégré et contrôlé via un modal qui s'ouvre et se ferme en fonction de l'état **isModalOpen**. Ce composant est spécifiquement lié à l'utilisateur connecté, avec l'ID récupéré depuis le stockage local (**localStorage**), démontrant une intégration personnalisée et sécurisée des notifications.

→ *Intégration du sous-composant **Notifications** avec gestion de l'état modal :*

```
<Notifications      isOpen={isModalOpen}      onClose={toggleModal}
userId={userId} />
```

Conclusion

Le composant **NavBar** est un exemple de la manière dont les techniques modernes de développement frontend peuvent être utilisées pour créer des composants riches en fonctionnalités et esthétiquement plaisants. Il démontre l'utilisation efficace des routes, des animations, de la gestion d'état et de l'interactivité dans une application React, tout en maintenant une attention constante à l'expérience utilisateur globale.

Les composants **SignupForm** et **LoginForm** de ce projet illustrent parfaitement l'utilisation de plusieurs outils et technologies frontales pour offrir une expérience utilisateur optimisée lors des processus d'inscription et de connexion. Ces composants sont cruciaux car ils gèrent la collecte des données utilisateur, la validation de ces données et l'interaction avec l'API backend pour authentifier les utilisateurs ou enregistrer de nouveaux comptes.

Caractéristiques Principales des Composants

1. Utilisation de Formik pour la Gestion des Formulaires : Formik est employé pour faciliter la gestion des états des formulaires, la validation des entrées avec Yup, et le traitement des soumissions. Ceci simplifie la création de formulaires robustes avec des validations intégrées, réduisant le code nécessaire et améliorant la maintenabilité.

→ *Exemple de validation avec Yup dans SignupForm :*

```
validationSchema={Yup.object({
  firstName: Yup.string().required("Required"),
  lastName: Yup.string().required("Required"),
  email: Yup.string().email("L'adresse e-mail saisie est invalide").required("Ce champ
est obligatoire."),
  password: Yup.string().min(6, "Votre mot de passe doit contenir au moins 6
caractères.").required("Ce champ est obligatoire."),)}}}
```



Le rendu sur navigateur



Connexion

Adresse Email

gre

L'adresse e-mail saisie est invalide

Mot de passe

Votre mot de passe doit contenir au moins 6 caractères.

Votre mot de passe doit être une combinaison d'au moins 6 lettres, chiffres et symboles.

☐ Se souvenir de moi

[Mot de passe oublié?](#)

Connexion

[Vous n'avez pas encore de compte? Créez-en un dès maintenant !](#)

2. Navigation avec React Router : L'utilisation de `useNavigate` de React Router permet de rediriger les utilisateurs après des actions réussies, comme à la suite d'une inscription ou d'une connexion, améliorant ainsi le flux utilisateur sans rechargements de page inutiles.

→ Exemple de redirection dans `LoginForm` :

```
navigate('/deadlines'); // Redirige l'utilisateur sur la page Deadline après une connexion réussie.
```

3. Utilisation d'Axios pour les requêtes API : Axios est utilisé pour envoyer des requêtes HTTP au backend. Cela inclut l'envoi des données du formulaire pour les opérations d'inscription et de connexion, en traitant les réponses et en gérant les erreurs.

→ Exemple de requête POST dans `SignupForm` :

```
axios.post(`${import.meta.env.VITE_BACKEND_URL}/users`, values)
```

Pour renforcer la flexibilité et simplifier les déploiements futurs, nous avons opté pour l'utilisation d'une variable d'environnement pour spécifier l'adresse du backend dans le frontend. Cette approche élimine la nécessité de répéter l'adresse du serveur à travers le code et facilite les modifications de configuration sans nécessiter de mises à jour directes du code source. En définissant cette adresse via une variable d'environnement, nous rendons notre application plus adaptable et plus facile à configurer pour différents environnements de déploiement.



```
call it in React with import.meta.env.VITE_BACKEND_URL  
VITE_BACKEND_URL=http://localhost:5000
```



Conclusion

Les composants **SignUpForm** et **LoginForm** sont des exemples de la manière dont les technologies modernes de développement frontend peuvent être harmonieusement combinées pour créer des expériences utilisateur fluides et sécurisées. Ils montrent l'efficacité de React et Formik dans la gestion des états de formulaire et des validations, dans l'élégance des animations avec Framer Motion, dans la souplesse de la navigation avec React Router, ainsi que dans la robustesse des interactions API avec Axios. Ces composants ne sont pas seulement fonctionnels mais sont également conçus pour être sécurisés, esthétiquement plaisants, alignant chaque aspect du design et de la fonctionnalité pour servir les utilisateurs de l'application.

Le composant **AboutContent** illustre comment une architecture frontale bien pensée et des technologies modernes peuvent être utilisées pour enrichir l'expérience utilisateur sur une page "À propos". Ce composant intègre des éléments visuels, des animations, une gestion de contenu dynamique, et des liens interactifs pour offrir une présentation complète de l'histoire, des valeurs, et de l'équipe du projet.

Caractéristiques Principales de AboutContent

1. Utilisation de Framer Motion pour les Animations : Framer Motion est utilisé pour ajouter des animations subtiles mais efficaces qui rendent le contenu plus engageant. Les images et les textes sont animés pour apparaître de manière plus dynamique et attirer l'attention de l'utilisateur.

→ Exemple d'animation pour l'image de l'histoire de l'entreprise :

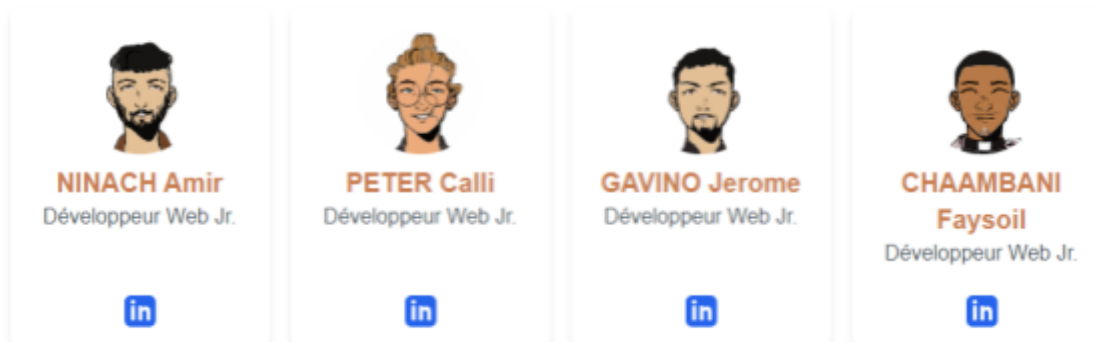
```
<motion.div variants={imageVariants} initial="hidden" animate="visible">
  <img src={history} alt="Notre Histoire" className="w-full
rounded-xl"/>
</motion.div>
```

2. Responsive Design et Accessibilité : Le design responsive est assuré par des styles flexibles et des grilles adaptatives qui permettent au contenu de s'ajuster parfaitement sur différents appareils, de l'ordinateur de bureau au mobile. L'accessibilité est également prise en compte avec des éléments interactifs clairement étiquetés, des contrastes de couleurs étudiés et des animations qui ne sont pas seulement esthétiques mais aussi informatives.

3. Composition de l'Équipe avec Liens Dynamiques : Chaque membre de l'équipe est présenté dans une carte interactive contenant sa photo, son rôle, et un lien vers son profil LinkedIn, permettant aux utilisateurs de se connecter facilement avec les membres de l'équipe.

Le rendu sur
navigateur

Notre Equipe





Exemple du code de carte de membre d'équipe :

```
<motion.div
  key={member.id}
  className="max-w-xs w-full sm:max-w-[200px] sm:h-[260px] min-h-[140px]
  bg-white rounded-lg shadow-md p-4 flex flex-col items-center m-2"
  initial={{ opacity: 0, y: 30 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ delay: 0.1 * index, duration: 0.5 }}
>
  <img src={member.image} alt="Profile" className="rounded-full w-20
  h-20 sm:w-24 sm:h-24 mb-2"/>
  <div className="text-center">
    <h2 className="text-lg sm:text-xl font-bold">{member.name}</h2>
    <p className="text-sm sm:text-base text-gray-600">{member.role}</p>
  </div>
  <a href={member.linkedin} target="_blank" rel="noopener noreferrer">
    <svg ...></svg>
  </a>
</motion.div>
```

4. Intégration du Composant FAQ : Le composant **FAQ** intégré offre une interaction utilisateur accrue en permettant aux visiteurs de cliquer sur des questions fréquentes pour en révéler les réponses. Cette fonctionnalité est animée pour permettre une meilleure expérience utilisateur, avec des mouvements doux qui révèlent les informations de manière progressive.

Conclusion

Le composant **AboutContent** combine les animations, le design responsive, l'interactivité et l'accessibilité pour fournir une expérience utilisateur riche et informative. Il démontre l'efficacité de React pour orchestrer un contenu complexe et dynamique tout en maintenant une interface propre et engageante. Chaque fonctionnalité est soigneusement conçue pour assurer non seulement l'esthétique mais aussi la fonctionnalité, rendant l'information non seulement accessible mais aussi agréable à explorer.

Le composant **FAQ** démontre l'utilisation efficace de React pour dynamiser l'affichage du contenu en utilisant des techniques de programmation moderne qui améliorent à la fois l'interactivité et l'accessibilité de l'application. En particulier, la méthode **.map()** qui joue un rôle clé dans le rendu dynamique des éléments de FAQ basés sur un tableau d'objets ; chaque objet représentant une question et une réponse.

Utilisation de **.map()** pour le Contenu Dynamique

Dans le composant **FAQ**, **.map()** est utilisé pour itérer sur un tableau de questions fréquemment posées. Cette méthode est idéale pour transformer des listes de données en éléments de React. Chaque question et sa réponse sont stockées dans un objet qui est ensuite transformé en composant visuel. Cette approche rend le code plus propre, plus facile à gérer et à maintenir.



Avantages de l'Utilisation de `.map()` pour le Rendu Dynamique

1. **Simplicité et efficacité** : `.map()` permet de créer des composants réactifs de manière concise et efficace. Chaque élément du tableau FAQ est transformé en un composant visuel sans avoir besoin de code impératif répétitif.
2. **Maintenance Facilitée** : La gestion de contenu dynamique via `.map()` facilite la maintenance et les mises à jour du contenu. Ajouter, modifier ou supprimer une FAQ est aussi simple que de mettre à jour le tableau `faqs`.
3. **Performance Optimisée** : React gère efficacement les listes rendues avec `.map()`, réutilisant les éléments DOM et minimisant les changements nécessaires lors de la mise à jour de la liste, ce qui améliore les performances globales.
4. **Accessibilité améliorée** : En utilisant des composants dynamiques qui réagissent aux interactions des utilisateurs, comme l'expansion et la rétraction des réponses FAQ, l'application devient plus interactive et accessible, répondant mieux aux besoins des utilisateurs.

Exemple de `.map()` dans FAQ:

```
{faqs.map((faq, index) => (  
  <div key={index} className="border rounded-lg shadow-md">  
    <button  
      className="w-full flex justify-between items-center p-4 font-semibold  
text-mg text-gray-900"  
      onClick={() => handleToggle(index)}>  
      <span>{faq.question}</span>  
      <span>{activeIndex === index ? "-" : "+"}</span>  
    </button>  
    <AnimatePresence initial={false}>  
      {activeIndex === index && (  
        <motion.div  
          initial="collapsed"  
          animate="open"  
          exit="collapsed"  
          variants={faqVariants}  
          className="p-4 pt-2 border-t text-base text-gray-700"  
          style={{ originY: 0 }}>  
            <p>{faq.answer}</p>  
          </motion.div>)}  
      </AnimatePresence> </div>))}
```


Comment puis-je m'inscrire sur DeadlineDrive ?

+

Qu'est-ce que le système de vote sur DeadlineDrive ?

+

Comment puis-je garder mes citations préférées ?

-

DeadlineDrive intègre un système de favoris pratique qui vous permet de sauvegarder jusqu'à 10 de vos citations préférées. Pour ajouter une citation à vos favoris, il suffit de cliquer sur l'icône en forme de cœur située à côté de la citation. Vous pouvez accéder à tout moment à votre liste de citations favorites depuis votre espace utilisateur.

Comment puis-je utiliser le calendrier interactif sur DeadlineDrive ?

+

Comment fonctionne les notifications que je reçois de DeadlineDrive ?

+

Conclusion

En conclusion, le composant `FAQ` illustre comment les techniques de développement front-end modernes, telles que l'utilisation de `.map()` pour le rendu de listes dynamiques, peuvent être utilisées pour améliorer l'expérience utilisateur en fournissant un contenu interactif et facilement accessible. Ce type de composant enrichit l'application en rendant les informations non seulement plus attrayantes mais aussi plus organisées et faciles à naviguer.

Le composant `CommentSection` illustre une gestion dynamique et interactive des commentaires au sein d'une application, mettant en œuvre des pratiques avancées avec React, notamment l'utilisation des hooks tels que `useState`, `useEffect`, et `useRef`, ainsi que des subtilités d'affichage conditionnel et de gestion des événements.

Fonctionnalités et Techniques Utilisées

1. Gestion des commentaires avec `useEffect` et `Axios` : `UseEffect` est utilisé pour charger les trois derniers commentaires relatifs à une citation spécifique dès que le composant est monté ou mis à jour. Cela permet de garder l'interface utilisateur synchronisée avec les données les plus récentes disponibles sur le serveur.

```
useEffect(() => {
  axios.get(`${import.meta.env.VITE_BACKEND_URL}/comments/by-quote/${quote.id}`)
    .then(response => {
      const lastThreeComments = response.data[0].slice(-3);
      setComments(lastThreeComments);
    })
    .catch(error => console.error("Error loading comments:", error));
}, [quote.id, comments]);
```



2. Affichage conditionnel pour les boutons d'édition et de suppression : L'interface permet aux utilisateurs de modifier ou supprimer leurs propres commentaires, ce qui est géré par un affichage conditionnel. Les boutons pour éditer et supprimer n'apparaissent que si l'utilisateur connecté est l'auteur du commentaire.

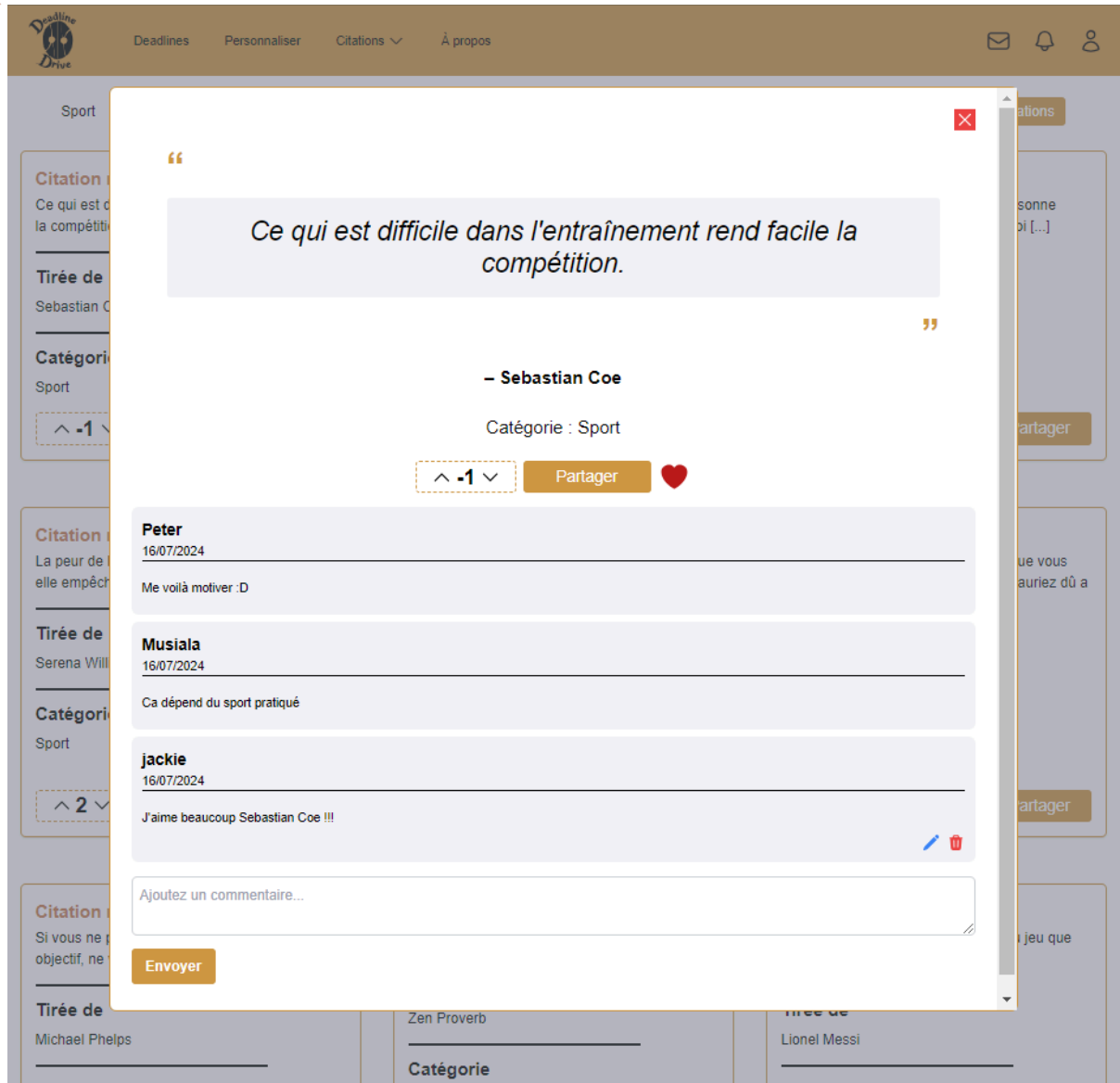
```
{parseInt(userId) === comment.id_user && (  
  <>  
    <button className="text-blue-500 hover:text-blue-700" onClick={() =>  
      handleEditComment(comment.id)}>  
      <PencilIcon className="w-5 h-5" />  
    </button>  
    <button className="text-red-500 hover:text-red-700" onClick={() =>  
      handleDeleteComment(comment.id)}>  
      <TrashIcon className="w-5 h-5" />  
    </button>  
  </>  
)}
```

3. Fermeture de la modal par clic extérieur : La gestion des clics en dehors de la modal pour fermer cette dernière est implémentée à l'aide d'un hook (`useRef`) et d'une fonction gérant les clics extérieurs. Cela améliore l'expérience utilisateur en permettant une interaction intuitive avec la modal.

```
const handleClickOutside = (event) => {  
  if (modalRef.current && !modalRef.current.contains(event.target)) {  
    onClose();  
  }  
};
```

4. Gestion de l'ajout de nouveaux commentaires : Les utilisateurs peuvent ajouter des commentaires via un champ de texte et un bouton "Envoyer". L'ajout d'un commentaire met à jour immédiatement l'affichage sans nécessiter de rechargement, grâce à une mise à jour locale de l'état après l'envoi du commentaire au serveur.

```
const handleAddComment = () => {  
  const commentData = {  
    content: newComment,  
    id_user: userId,  
    id_quote: quote.id,  
  };  
  axios.post(`${import.meta.env.VITE_BACKEND_URL}/comments`,  
    commentData)  
    .then(response => {  
      const updatedComments = [...comments, {  
        ...commentData,  
        username: getUserFirstNameFromLocalStorage(),  
        created_at: new Date().toISOString(),  
      }].slice(-3);  
      setComments(updatedComments);  
      setNewComment("");  
    })  
    .catch(error => console.error("Failed to add comment:", error));  
};
```



Conclusion

Le **CommentSection** démontre la puissance des hooks de React pour une gestion efficace des données dynamiques et des interactions utilisateurs dans une application moderne. Les fonctionnalités avancées telles que l'affichage conditionnel, la gestion des événements externes, et les interactions en temps réel avec la base de données via Axios, garantissent une expérience utilisateur fluide et intuitive. Ces pratiques illustrent bien comment les applications web modernes peuvent gérer de manière efficace et élégante les interactions complexes entre l'utilisateur et l'interface.

Le composant **NotificationsComponent** offre une gestion élégante des notifications utilisateur dans une application React, illustrant l'efficacité des hooks React pour le contrôle de l'état et des effets secondaires. Ce composant sert de modal pour afficher et interagir avec les notifications de l'utilisateur, employant **useState**, **useEffect**, et **AnimatePresence** pour une expérience utilisateur fluide et réactive.



Principales fonctionnalités et techniques utilisées

1. Gestion dynamique des notifications avec `useState` et `useEffect` : Le hook `useEffect` est utilisé pour charger les notifications de l'utilisateur lorsque la modal est ouverte. Ce mécanisme assure que les données sont toujours à jour et réduit les charges inutiles lorsque la modal n'est pas active.

```
useEffect(() => {  
  if (isOpen && userId) {  
    fetchNotifications();  
  }}, [isOpen, userId]);
```

Cette approche permet de maintenir les notifications synchronisées avec les événements de l'application, en les rechargeant à chaque ouverture de la modal.

2. Affichage dynamique et gestion des états de lecture : Les notifications sont affichées avec une icône qui change selon l'état de lecture, permettant aux utilisateurs de voir rapidement quelles notifications ont été lues. Un clic sur l'icône non lue (`EyeSlashIcon`) déclenche une mise à jour de son état en utilisant une requête PUT au backend.

```
{notification.is_read ? (  
  <EyeIcon className="h-6 w-6 text-green-500" />  
) : (  
  <EyeSlashIcon  
    className="h-6 w-6 text-red-500 cursor-pointer"  
    onClick={() => markAsRead(notification.id)}  
  />  
)}
```

3. Gestion des interactions utilisateur avec `AnimatePresence` : `AnimatePresence` est utilisé pour animer l'entrée et la sortie de la modal, améliorant l'expérience visuelle et rendant l'interaction plus naturelle et moins abrupte.

```
<AnimatePresence>  
  {isOpen && (  
    <motion.div  
      className="fixed z-10 inset-0 bg-gray-500 bg-opacity-75 flex justify-center  
        items-center"  
      initial={{ opacity: 0 }}  
      animate={{ opacity: 1 }}  
      exit={{ opacity: 0 }}  
      onClick={onClose}  
    >  
      { /* Modal Content */ }  
    </motion.div>  
  )}</AnimatePresence>
```



4. Suppression de notifications : Les utilisateurs peuvent supprimer des notifications via une icône de poubelle, qui envoie alors une requête DELETE au backend pour supprimer la notification de manière permanente. Cette action est immédiatement reflétée dans l'interface utilisateur par une mise à jour de l'état local.

```
<TrashIcon  
  className="h-6 w-6 text-red-500 cursor-pointer ml-2"  
  onClick={() => deleteNotification(notification.id)}  
/>
```

Notifications

Citation du jour : Il n'y a pas de hasard, il n'y a que des rendez-vous. - Le Fabuleux Destin d'Amélie Poulain

Reçu le 16/07/2024



Citation du jour : L'espoir est l'ancre de l'âme. - Saint Augustin

Reçu le 16/07/2024



Citation du jour : Le bonheur n'est réel que lorsqu'il est partagé. - Jon Krakauer

Reçu le 15/07/2024



Citation du jour : L'espoir est la lumière qui nous guide dans les moments les plus sombres. - Emma (The Promised Neverland)

Reçu le 12/07/2024



Citation du jour : La persévérance est la clé du succès. Il ne faut jamais abandonner ses rêves. - Sakura Haruno (Naruto)

Reçu le 11/07/2024



Fermer

Conclusion

Le `NotificationsComponent` démontre l'utilisation avancée des hooks React pour créer une expérience utilisateur riche et interactive autour des notifications. L'utilisation de techniques modernes de gestion d'état et d'animation permet non seulement d'assurer une interface réactive mais aussi de fournir des feedbacks visuels et fonctionnels clairs aux utilisateurs. Ces pratiques sont essentielles pour des applications modernes visant à optimiser l'engagement et la satisfaction utilisateur.



Le **Footer** et le **ScrollToTopButton** sont deux composants cruciaux qui améliorent la navigation et l'expérience utilisateur dans une application web. Ces composants servent à fournir des informations essentielles et à faciliter l'accès à diverses sections du site de manière intuitive.

Footer

Le **Footer** est stylisé avec une couleur d'arrière-plan personnalisée et contient des liens vers des pages importantes comme la "Politique de confidentialité" et les "Mentions légales". Cette section est non seulement utile pour l'orientation de l'utilisateur mais aussi pour renforcer la transparence et la confiance en fournissant un accès facile à des informations légales.

```
<footer className="bg-custom-main-orange text-center py-4">
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
    <div className="flex flex-col md:flex-row justify-center items-center space-y-2
md:space-y-0 md:space-x-4">
      <Link to="/legal-notice" className="text-custom-black font-bold
hover:text-gray-200">
        Politique de confidentialité
      </Link>
      <span className="text-custom-black">
        Deadline Drive 2024@ Tous droits réservés.
      </span>
      <Link to="/privacy-policy" className="text-custom-black font-bold
hover:text-gray-200">
        Conditions d'utilisation
      </Link>
    </div>
  </div>
</footer>
```

ScrollToTopButton

Le **ScrollToTopButton** utilise un hook **useState** pour gérer sa visibilité basée sur la position de défilement de la page. Ce bouton permet aux utilisateurs de revenir en haut de la page avec une animation douce, ce qui est particulièrement utile dans les pages à contenu dense.

```
const ScrollToTopButton = () => {
  const [isVisible, setIsVisible] = useState(false);
  useEffect(() => {
    const toggleVisibility = () => {
      if (window.pageYOffset > 300) {setIsVisible(true);
    } else {setIsVisible(false);}};window.addEventListener('scroll', toggleVisibility);
    return () => { window.removeEventListener('scroll', toggleVisibility);}}, []);
  const scrollToTop = () => {
    window.scrollTo({
      top: 0,
      behavior: 'smooth'});};
  return ( <button onClick={scrollToTop} aria-label="Retour en haut de page"
className={`fixed bottom-4 right-4 z-50 bg-butterscotch hover:bg-caramel text-white
font-bold py-2 px-4 rounded-full shadow-lg hover:shadow-xl transition ease-in-out
duration-300 ${isVisible ? 'block' : 'hidden'}`}><ChevronDoubleUpIcon className="h-6
w-6" /> </button>);};
```




Conclusion

Ces composants sont des éléments fondamentaux pour une navigation efficace et une expérience utilisateur agréable, fournissant à la fois une fonctionnalité pratique et renforçant la crédibilité du site. Ils démontrent l'attention portée aux détails dans la conception de l'interface utilisateur, favorisant une interaction positive avec l'application.

La construction du front-end de cette application a été méticuleusement orchestrée pour offrir une expérience utilisateur fluide et engageante, grâce à l'utilisation de technologies modernes et de pratiques de développement avancées. Voici les points clés qui ressortent de la réalisation de ce projet :

1. **Utilisation de React et React Router** : L'application tire parti de React pour construire une interface utilisateur réactive et dynamique, avec React Router pour gérer la navigation entre les pages sans recharger la page, ce qui rend l'expérience utilisateur rapide et fluide.
2. **Animations et interactions visuelles** : L'usage de Framer Motion pour les animations enrichit l'interaction utilisateur, apportant une touche esthétique et fonctionnelle qui guide et réjouit les utilisateurs à travers les transitions fluides et les feedbacks visuels.
3. **Composants réutilisables et modulaires** : Les composants comme `Footer`, `ScrollToTopButton`, la `NavBar` et les formulaires gérés avec Formik montrent une approche de développement modulaire. Cela non seulement optimise la cohérence et la maintenance du code mais favorise également la réutilisabilité à travers l'application.
4. **Gestion avancée des états et des données** : L'utilisation de hooks tels que `useState` et `useEffect` pour gérer les états locaux et les effets secondaires dans les composants tels que `NotificationsComponent` et `CommentSection` garantit que les données sont à jour et synchronisées avec l'interface utilisateur.
5. **Responsivité et accessibilité** : Grâce à l'utilisation de Tailwind CSS, l'application maintient une interface cohérente et accessible sur divers dispositifs, assurant que tous les utilisateurs, quel que soit leur appareil ou leurs besoins en accessibilité, peuvent naviguer facilement.
6. **Sécurité et gestion des utilisateurs** : L'intégration de pratiques de sécurité dans les formulaires de connexion et d'inscription, avec la gestion des JWT pour l'authentification, montre un engagement envers la protection des données utilisateurs et l'intégrité de l'application.
7. **Interaction avec le backend** : L'utilisation d'axios pour les requêtes API assure une intégration fluide avec le backend, permettant des opérations CRUD efficaces. En combinaison avec les `useState`, on obtient une synchronisation en temps réel des données utilisateur et des notifications.

En somme, le développement du front-end de cette application démontre une attention particulière à la qualité de code, à l'expérience utilisateur, et à l'architecture de l'application. Ces éléments sont essentiels pour assurer que l'application ne soit pas seulement fonctionnelle mais aussi plaisante à utiliser, sécurisée, et facile à maintenir et à évoluer.



Accessibilité

L'accessibilité est une priorité dans notre projet, visant à garantir que notre application est utilisable par tous, y compris les personnes en situation de handicap. Nous avons intégré des pratiques d'accessibilité, telles que l'utilisation de balises ARIA, des contrastes de couleurs adéquats et des alternatives textuelles pour les images. Ces mesures permettent d'améliorer la navigation et l'interaction pour les utilisateurs avec des aides techniques.

Exemple de code :

```
<button aria-label="Fermer le menu" onClick={handleClose}>
  <span className="sr-only">Fermer</span>
  <XMarkIcon className="w-6 h-6" />
</button>
```

Utilisation de Lighthouse

Lighthouse a été utilisé pour évaluer les performances, l'accessibilité, les meilleures pratiques et le référencement (SEO) de l'application DeadlineDrive. Les résultats de cette analyse montrent des scores élevés dans toutes les catégories, indiquant une application optimisée et bien structurée.

Cependant, l'utilisation de React.js présente certaines limitations pour un référencement optimal, car le contenu généré côté client peut ne pas être immédiatement visible pour les moteurs de recherche. Pour pallier cela, des améliorations futures seront apportées, notamment en ajustant les couleurs pour une meilleure visibilité et en optimisant les pratiques de référencement pour atteindre un public plus large.



There were issues affecting this run of Lighthouse:

- Clearing the origin data timed out. Try auditing this page again and file a bug if the issue persists.
- Clearing the browser cache timed out. Try auditing this page again and file a bug if the issue persists.



Performance

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator.](#)

▲ 0-49 ■ 50-89 ● 90-100





Éco-Conception

L'éco-conception vise à minimiser l'impact environnemental des applications web. Dans les versions futures de notre projet, nous prévoyons d'optimiser la taille des images, de réduire le nombre de requêtes, et de charger uniquement les composants nécessaires. Par exemple, l'utilisation de `React.lazy` permet de charger les composants de manière asynchrone et uniquement lorsque c'est nécessaire, réduisant ainsi le temps de chargement initial et économisant des ressources.

```
// Exemple d'utilisation de React.lazy dans notre projet
const NotificationsComponent = React.lazy(() =>
import('./NotificationsComponent'));

// Utilisation React.lazy sur le composant Notifications
function App() {
  return (
    <div>
      <React.Suspense fallback=<div>Loading...</div>>
        <NotificationsComponent />
      </React.Suspense>
    </div>
  );
}
```

Ces optimisations contribueront à rendre l'application plus rapide, plus efficace, et plus respectueuse de l'environnement.

Affichage Conditionnel

```
      <button aria-label="Modifier le commentaire"
        className="text-blue-500 hover:text-blue-700"
        onClick={() => handleEditComment(comment.id)}
      >
        {parseInt(userId) === comment.id_user && <PencilIcon
className="w-5 h-5" />}
      </button>
      <button aria-label="Supprimer le commentaire"
        className="text-red-500 hover:text-red-700"
        onClick={() => handleDeleteComment(comment.id)}
      >
        {parseInt(userId) === comment.id_user && <TrashIcon
className="w-5 h-5" />}
      </button>
```

Dans cet extrait, les icônes pour éditer et supprimer un commentaire sont conditionnellement affichées uniquement si l'ID de l'utilisateur actuellement connecté correspond à l'ID de l'utilisateur qui a posté le commentaire. Cela garantit que les utilisateurs ne peuvent modifier ou supprimer que leurs propres commentaires, ajoutant ainsi une couche de sécurité et de personnalisation à l'interface utilisateur.



c. Back-end

Dans ce projet, nous avons adopté une approche de programmation orientée objet (POO) dans les modèles et mis en œuvre le modèle de conception MVC (Model-View-Controller) pour structurer et gérer de manière efficace le développement du backend en JavaScript et du frontend avec React.js. Ce cadre organisationnel a été renforcé par l'intégration d'opérations CRUD (Create, Read, Update, Delete) pour interagir avec la base de données. Voici une analyse détaillée des différentes composantes et de leur interaction.

Programmation Orientée Objet (POO)

Le projet utilise les principes de la POO, notamment l'encapsulation, l'héritage et le polymorphisme, l'abstraction pour structurer le code de manière modulaire et réutilisable.

La définition de classes et d'objets permet de modéliser des entités réelles au sein de l'application.

- **Encapsulation** : Regroupe les données et les méthodes qui les manipulent au sein de classes, protégeant ainsi l'intégrité des données.
- **Héritage** : Permet à une classe d'hériter des attributs et méthodes d'une autre, facilitant la réutilisation du code et la hiérarchisation.
- **Polymorphisme** : Permet à des méthodes de se comporter différemment en fonction de l'objet qui les appelle, offrant une flexibilité dans l'interaction avec les objets.
- **Abstraction** : Cache les détails complexes et montre uniquement les fonctionnalités essentielles, simplifiant ainsi la programmation et la maintenance.

Ces principes de POO permettent de structurer le code de manière modulaire, réutilisable et facile à gérer.

AbstractManager et CommentManager : La classe **AbstractManager** sert de base pour la gestion des interactions avec la base de données pour différentes tables. Elle encapsule les méthodes communes telles que **find**, **findAll**, et **delete**, qui sont ensuite héritées par des gestionnaires plus spécifiques tels que **CommentManager** et **NotificationManager**.

```
class AbstractManager {
  constructor({ table }) {
    this.table = table;
  }
  find(id) {
    return this.database.query(`SELECT * FROM ${this.table} WHERE id = ?`, [id]);
  }
  findAll() {
    return this.database.query(`SELECT * FROM ${this.table}`);
  }
  delete(id) {
    return this.database.query(`DELETE FROM ${this.table} WHERE id = ?`, [id]);
  }
  setDatabase(database) {
    this.database = database;
  }
}
```

La classe **CommentManager** hérite de **AbstractManager** et étend cette dernière en ajoutant des méthodes spécifiques pour gérer les commentaires, telles que **insert**, **update**, et **findByQuoteId**, illustrant l'héritage et le polymorphisme dans la POO.



```
class CommentManager extends AbstractManager {
  constructor() {
    super({ table: 'comment' });}
  insert(comment) {
    const { content, id_user, id_quote } = comment;
    const created_at = new Date();
    const updated_at = new Date();
    return this.database.query(
      `INSERT INTO ${this.table} (content, created_at, updated_at, id_user, id_quote) VALUES (?, ?, ?, ?, ?)`,
      [content, created_at, updated_at, id_user, id_quote]
    );
  }
  update(comment) {
    const fields = [];
    const values = [];
    for (const field of ['content', 'updated_at']) {
      if (comment[field] !== undefined) {
        fields.push(`${field} = ?`);
        values.push(comment[field]);
      }
    }
    if (fields.length === 0) {
      throw new Error('No fields have been modified.');
    }
    const query = `UPDATE ${this.table} SET ${fields.join(', ')} WHERE id = ?`;
    values.push(comment.id);
    return this.database.query(query, values);
  }
  findByQuoteId(quoteId) {
    return this.database.query(`SELECT * FROM ${this.table} WHERE id_quote = ?`, [quoteId]);
  }
}
```

Modèle MVC

Le modèle MVC se compose de trois composants principaux :

Modèle (Model): Le modèle gère toutes les opérations de base de données. Il interagit directement avec la base de données pour exécuter des requêtes SQL et renvoie les résultats au contrôleur. Par exemple, `CommentManager` est un modèle qui contient la logique pour gérer les commentaires dans la base de données.

Vue (View): La vue est en quelque sorte gérée par React.js dans le frontend, qui consomme des données envoyées par le backend sous forme de JSON et met à jour l'interface utilisateur en réponse aux changements de données. Par exemple, une View typique du modèle MVC aurait été le moteur de template Twig de symphony qui renvoie en temps réel des pages HTML, CSS.

Contrôleur (Controller): Les contrôleurs traitent les demandes entrantes, délèguent la gestion des données aux modèles, et renvoient les réponses appropriées aux clients. Ils servent de pont entre le modèle et la vue. Par exemple, `commentControllers.js` gère les opérations CRUD pour les commentaires.



Opérations CRUD pour les Commentaires

Create (Ajouter un commentaire): L'ajout de commentaires est géré par la méthode `insert` de `CommentManager` et la fonction `add` du contrôleur. Cette fonction vérifie d'abord les erreurs de validation avant d'insérer le commentaire dans la base de données.

```
const add = async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  const comment = req.body;
  try {
    const result = await models.comment.insert(comment);
    res.location(`/comments/${result.insertId}`).sendStatus(201);
  } catch (error) {
    console.error(error);
    res.status(400).send({ error: error.message });
  }
};
```

Read (Lire les commentaires): La lecture des commentaires est gérée par les méthodes `find` et `findById` de `CommentManager`, et les fonctions `read` et `findById` du contrôleur.

```
const read = async (req, res) => {
  try {
    const comment = await models.comment.find(req.params.id);
    if (!comment) {
      res.sendStatus(404);
    } else {
      res.send(comment);
    }
  } catch (error) {
    console.error(error);
    res.sendStatus(500);
  }
};
```

Update (Modifier un commentaire): La modification des commentaires est gérée par la méthode `update` de `CommentManager` et la fonction `edit` du contrôleur. Cette fonction vérifie d'abord les erreurs de validation avant de mettre à jour le commentaire dans la base de données.

```
const edit = async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  const comment = { ...req.body, id: parseInt(req.params.id, 10) };
  try {
    await models.comment.update(comment);
    res.sendStatus(204);
  } catch (error) {
    console.error(error);
    res.status(400).send({ error: error.message });
  }
};
```




Delete (Supprimer un commentaire): La suppression des commentaires est gérée par la méthode `delete` de `CommentManager` et la fonction `destroy` du contrôleur.

```
const destroy = async (req, res) => {
  try {
    const result = await models.comment.delete(req.params.id);
    if (result.affectedRows === 0) {
      res.sendStatus(404);
    } else {
      res.sendStatus(204);
    }
  } catch (error) {
    console.error(error);
    res.sendStatus(500);
  }
};
```

Notifications

Des opérations similaires ont été réalisées pour les notifications, avec une particularité dans l'opération `update` qui marque une notification comme lue en mettant à jour un champ boolean `is_read`.

```
const markAsRead = async (req, res) => {
  try {
    const result = await models.notification.markAsRead(req.params.id);
    if (result.affectedRows > 0) {
      res.sendStatus(200);
    } else {
      res.sendStatus(404);
    }
  } catch (error) {
    console.error(error);
    res.status(500).send("Error updating notification status");
  }
};
```

Gestion de la Connexion et de l'Inscription

La gestion de la connexion et de l'inscription est assurée par le hachage de mots de passe et la génération de tokens JWT pour sécuriser les sessions des utilisateurs.

Hachage des Mots de Passe: Lors de l'inscription, le mot de passe de l'utilisateur est haché avant d'être stocké dans la base de données.

```
const hashPassword = async (req, res, next) => {
  try {
    const hashedPassword = await argon2.hash(req.body.password);
    req.body.password = hashedPassword;
    next();
  } catch (err) {
    console.error('Error hashing password:', err);
    res.sendStatus(500);
  }
};
```



Vérification des Mots de Passe et Génération de Tokens: Lors de la connexion, le mot de passe de l'utilisateur est haché et ensuite comparé au mot de passe haché stocké dans la base de données. Si les deux mot de passe haché sont identiques, un token JWT est généré. A savoir que le mot de passe haché

```
const verifyPassword = async (req, res, next) => {
  try {
    const isVerified = await argon2.verify(req.user.password, req.body.password);
    if (isVerified) {
      const payload = { sub: req.user.id };
      const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: "99h" });
      delete req.user.password;
      res.send({ user: req.user, token: token });
    } else {
      res.sendStatus(401);
    }
    next();
  } catch (err) {
    console.error('Error verifying password:', err);
    res.sendStatus(500);
  }
};
```

Services Cron et Gestion des Citations

Pour améliorer l'engagement des utilisateurs, j'ai mis en place un système d'envoi quotidien de notifications de citations en utilisant un service cron et un service de gestion des citations. Ce dernier est automatisé grâce à la bibliothèque Node-Cron, qui permet de programmer des tâches à exécuter à des intervalles réguliers. Dans notre cas, elle garantit que les utilisateurs reçoivent chaque jour une citation inspirante.

Service Cron: Le service cron est planifié pour s'exécuter toutes les 12 heures et envoyer des notifications de citations à chaque utilisateur.

```
const cron = require('node-cron');
const { sendDailyNotification } = require('./quoteService');

function scheduleDailyNotifications() {
  cron.schedule('0 */12 * * *', () => {
    console.log('Running daily notification job every 12 hours');
    sendDailyNotification();
  }, {
    scheduled: true,
    timezone: "Europe/Paris"
  });
}
module.exports = { scheduleDailyNotifications };
```



Service de Gestion des Citations: Le service de gestion des citations récupère la citation du jour et envoie une notification à chaque utilisateur.

```
const axios = require('axios');
const models = require("../models");

const API_BASE_URL = 'http://localhost:5000';

async function fetchDailyQuote() {
  try {
    const response = await axios.get(`${API_BASE_URL}/daily-quote`);
    if (response.data) {
      return response.data;
    } else {
      throw new Error('No quote available');
    }
  } catch (error) {
    console.error("Error fetching quote from backend:", error);
    throw new Error('Failed to fetch quote');
  }
}

async function sendNotificationsToUsers(quote) {
  const users = await models.user.findAll();
  users[0].forEach(async (user) => {
    try {
      await models.notification.create(user.id, quote.id);
      console.log(`Notification sent to user ${user.id} for quote ${quote.id}`);
    } catch (error) {
      console.error(`Failed to send notification to user ${user.id}:`, error);
    }
  });
}

exports.sendDailyNotification = async () => {
  try {
    const quote = await fetchDailyQuote();
    await sendNotificationsToUsers(quote);
    console.log("Daily quote sent successfully.");
  } catch (error) {
    console.error("Failed to send daily quote:", error);
  }
};
```

Conclusion

L'architecture MVC combinée avec les principes de la POO et les pratiques de sécurité robustes fournit une fondation solide pour le développement des composants d'accès aux données SQL et des composants métier côté serveur. En utilisant des outils comme **express-validator** pour valider les entrées et des tokens JWT pour sécuriser les routes, nous avons assuré la sécurité et l'efficacité de l'application. La mise en œuvre de services cron et de notifications a amélioré l'engagement des utilisateurs, rendant l'application plus interactive et utile. Ce cadre garantit une maintenance et une évolutivité aisées, essentielles pour répondre aux besoins changeants de l'application et de ses utilisateurs.



API RESTful

Dans notre architecture backend, nous avons implémenté une **API RESTful** qui facilite l'interaction entre le frontend développé en React.js et notre serveur. Cette API est essentielle pour assurer une communication efficace et structurée à travers des requêtes HTTP standard. Chaque contrôleur dans notre modèle MVC expose des endpoints REST qui acceptent des requêtes pour effectuer les opérations CRUD sur la base de données via des modèles spécifiques. (Documentation complète annexe page 68-70)

Endpoints et Méthodes HTTP :

- **GET** pour lire des données (ex. /comments, /comments/{id} pour les commentaires).
- **POST** pour créer de nouvelles entités (ex. /comments pour ajouter un commentaire).
- **PUT** ou **PATCH** pour mettre à jour des entités existantes (ex. /comments/{id} pour modifier un commentaire).
- **DELETE** pour supprimer des entités (ex. /comments/{id} pour supprimer un commentaire).

Ces endpoints sont conçus pour être intuitifs et alignés avec les principes de design des API REST, facilitant ainsi l'intégration avec d'autres systèmes et le développement frontend. Les contrôleurs gèrent les requêtes entrantes, invoquent les méthodes adéquates des modèles pour accéder ou manipuler les données, et renvoient des réponses sous forme de JSON, permettant une interopérabilité totale avec le frontend React.js.

Sécurité et Authentification : La sécurité de notre API REST est renforcée par l'utilisation de tokens JWT (JSON Web Tokens) pour l'authentification et l'autorisation des requêtes. Chaque requête nécessitant une vérification d'identité est interceptée par des middlewares qui valident le token fourni avant de permettre l'accès aux ressources protégées.

Gestion des Erreurs : La robustesse de notre API est également assurée par une gestion systématique des erreurs, où chaque réponse d'erreur est retournée avec un code de statut HTTP approprié et un message d'erreur descriptif, aidant ainsi les développeurs du frontend à gérer ces situations du côté client efficacement.

Catégorie	Code	Signification
1XX - Informationnel	100	Continuer
2XX - Succès	200	OK
	201	Crée
	204	Pas de contenu
3XX	302	Trouvé (Redirection temporaire)
4XX - Erreur client	400	Mauvaise requête
	401	Non autorisé
	404	Non trouvé
5XX - Erreur serveur	500	Erreur interne du serveur



Documentation : La documentation en anglais de l'API Restful, réalisée avec Postman, offre une vue détaillée de tous les endpoints, les méthodes HTTP, les réponses attendues ainsi que les structures de données, ce qui simplifie l'intégration et le test par les développeurs. Vous pouvez retrouver le read.me spécialement créé pour documenter notre API RESTful sur github directement sur le lien du repository.

→ <https://github.com/KD63799/deadlinedrive/blob/dev/backend/API.md>

Sécurité

1. Base de données

La sécurité des données est primordiale dans la gestion de notre base de données hébergée sur Alwaysdata. Nous prenons plusieurs mesures pour assurer la protection des informations personnelles, en particulier le nom, prénom, et email de nos utilisateurs.

Conformité au RGPD : Nous avons adapté nos processus pour être en conformité avec le Règlement Général sur la Protection des Données (RGPD). Cela inclut l'obtention du consentement explicite des utilisateurs pour la collecte et le traitement de leurs données, la possibilité pour les utilisateurs de consulter, modifier, et supprimer leurs informations personnelles, et la mise en œuvre de mesures strictes pour protéger leurs données contre tout accès non autorisé.

Droits d'accès à la base de données : L'accès à la base de données est strictement régulé. Seuls les membres autorisés de l'équipe de développement et de gestion de la base de données ont les droits nécessaires pour accéder aux données, basés sur leur rôle dans le projet. Ces droits sont révisés régulièrement pour s'assurer qu'ils sont toujours appropriés.

Sauvegarde de la base de données : Des sauvegardes automatiques de la base de données sont effectuées quotidiennement pour prévenir toute perte de données. Ces sauvegardes sont stockées dans des emplacements sécurisés et sont accessibles uniquement par le personnel autorisé. En cas de défaillance ou de perte de données, nous pouvons rapidement restaurer la base de données à partir de ces sauvegardes, minimisant ainsi les perturbations de service. (annexe page 71)

En hébergeant notre base de données sur Alwaysdata, nous bénéficions également des protocoles de sécurité avancés et de l'infrastructure robuste du fournisseur, qui incluent le chiffrement des données en transit et au repos, ainsi que la surveillance continue des systèmes pour détecter et réagir aux menaces de sécurité de manière proactive.

2. Formik

Formik est une bibliothèque pour React qui simplifie la gestion des formulaires, y compris la validation des données. Il s'intègre parfaitement avec Yup pour la validation de schéma.

```
const PasswordRecoverySchema = Yup.object().shape({
  email: Yup.string()
    .email('Adresse email invalide')
    .required('L'email est requis'), });
```

- Formik gère l'état du formulaire et sa soumission.
- Yup est utilisé pour définir le schéma de validation des données.
- Le formulaire inclut un champ de saisie pour l'adresse email, avec validation en temps réel des erreurs.



3.Express Validator

Express Validator est une bibliothèque middleware pour les applications Node.js utilisant le framework Express.js, qui permet de valider et de normaliser les données des requêtes HTTP. Elle garantit que les informations reçues par le serveur sont conformes aux attentes et aux exigences de l'application. En validant et en nettoyant les entrées utilisateur avant qu'elles ne soient traitées ou insérées dans une base de données, Express Validator aide à prévenir les problèmes de sécurité tels que les injections SQL et les attaques XSS.

Exemple d'utilisation de Express Validator dans un endpoint de commentaires :

```
const { body, validationResult } = require('express-validator');

// Route pour la mise à jour d'un commentaire existant avec validation
utilisant Express Validator
router.put('/comments/:id', [
  body('content').optional().trim().isLength({ min: 1 }).withMessage('Le contenu ne peut pas être vide'),
  body('content').optional().isLength({ max: 500 }).withMessage('Le contenu est trop long, maximum 500 caractères'),
  body('id_user').optional().isNumeric().withMessage('L\'ID de l\'utilisateur doit être numérique'),
  body('id_quote').optional().isNumeric().withMessage('L\'ID de la citation doit être numérique si fourni')
], commentControllers.edit);

// Fonction du point de terminaison pour éditer un commentaire
const edit = async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    // Retourner une erreur 400 avec les détails des erreurs si la validation échoue
    return res.status(400).json({ errors: errors.array() });
  }
  const comment = { ...req.body, id: parseInt(req.params.id, 10) };
  try {
    // Mise à jour du commentaire dans la base de données
    await models.comment.update(comment);
    res.sendStatus(204); // Envoyer un statut 204 No Content en cas de succès
  } catch (error) {
    console.error(error);
    res.status(400).send({ error: error.message }); // Envoyer une erreur 400 si la mise à jour échoue
  }
};
```




4. Requête SQL Préparée

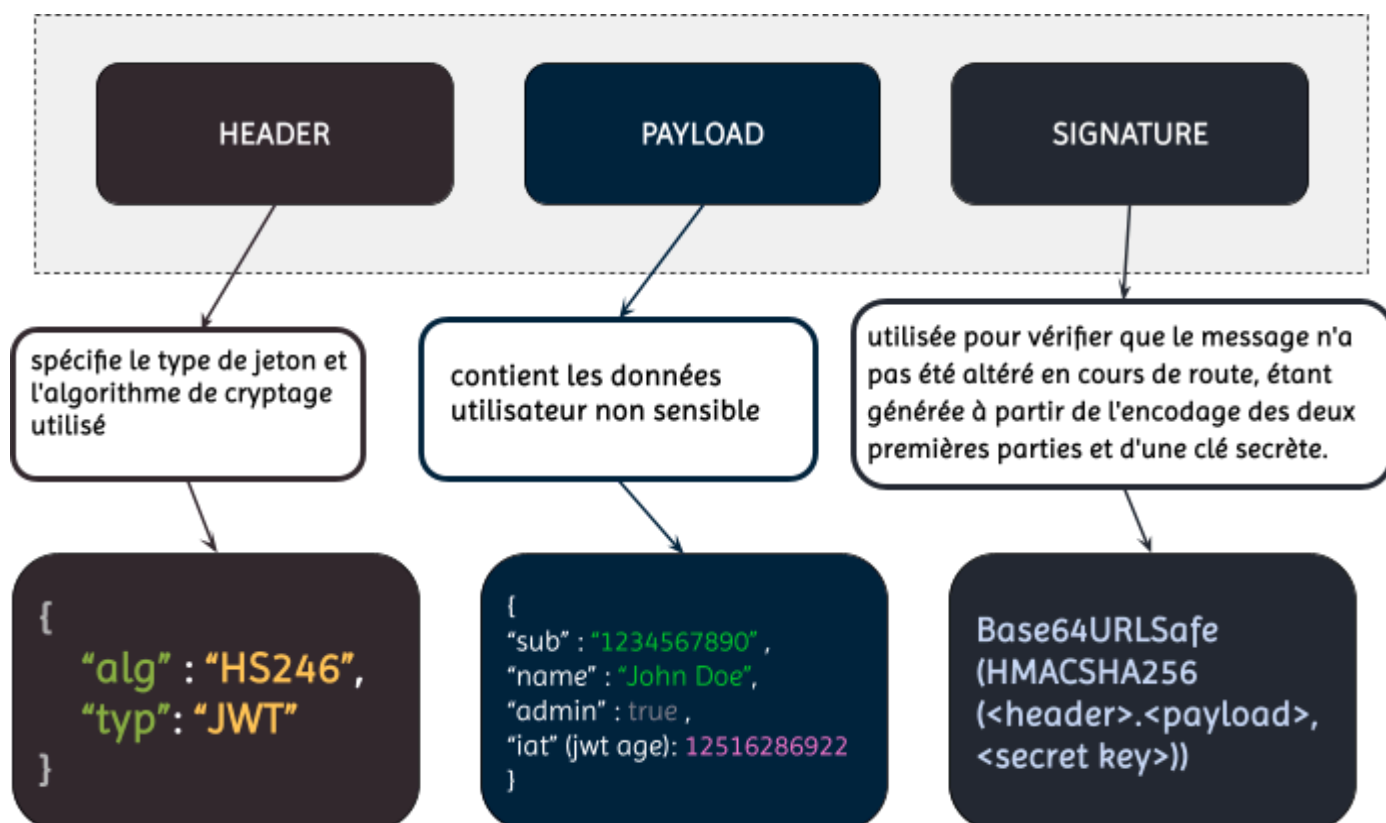
Les requêtes SQL préparées sont une technique de base de données pour exécuter des instructions SQL avec des paramètres, permettant ainsi de prévenir les injections SQL.

Exemple de requête SQL préparée :

```
// Ajoute un commentaire dans la base de données
insert(comment) { let { content, id_user, id_quote } = comment;
const created_at = new Date();
const updated_at = new Date();

// Exécute une requête SQL préparée pour ajouter un commentaire dans la base de données
return this.database.query(
  `INSERT INTO ${TABLE_NAME} (content, created_at, updated_at, id_user, id_quote)
VALUES (?, ?, ?, ?, ?)`,
  [content, created_at, updated_at, id_user, id_quote]
);
}
```

Le JWT token





5. Requête HTTP avec Token

Les requêtes HTTP intégrant des tokens de sécurité, comme les JWT (JSON Web Tokens), renforcent la sécurité en assurant que les requêtes sont authentifiées et autorisées.

Exemple d'envoi d'une requête HTTP avec un token JWT :

```
// Middleware pour vérifier le token JWT et attacher l'utilisateur à la requête
const verifyToken = async (req, res, next) => {
  try {
    // Récupérer l'en-tête d'autorisation de la requête
    const authorizationHeader = req.get("Authorization");
    if (!authorizationHeader) {
      throw new Error("L'en-tête d'autorisation est manquant");
    }
    // Séparer l'en-tête d'autorisation pour obtenir le type et le token
    const [type, token] = authorizationHeader.split(" ");
    if (type !== "Bearer") {
      throw new Error("Le type de l'en-tête d'autorisation n'est pas 'Bearer'");
    }
    // Vérifier le token JWT en utilisant le secret
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.payload = payload;
    console.log('Payload:', payload); // Afficher le payload pour débogage
    // Utiliser le payload pour obtenir l'ID de l'utilisateur et le chercher dans la
    base de données
    const user = await models.user.findById(payload.sub);
    if (!user) {
      return res.status(404).send('Utilisateur non trouvé');
    }
    next(); // Passer au middleware suivant
  } catch (err) {
    console.error('Erreur lors de la vérification du token :', err);
    res.sendStatus(401); // Envoyer un statut 401 Unauthorized si la vérification du
    token échoue
  }
};
```

Conclusion

L'adoption de pratiques sécurisées et de bibliothèques robustes dans le développement de l'application assure une fondation solide pour maintenir l'intégrité et la sécurité des données. En utilisant des techniques telles que les validations de formulaire avec Express Validator et Formik, les requêtes SQL préparées, et la gestion sécurisée des sessions avec des tokens JWT, nous avons non seulement renforcé la sécurité de l'application, mais également amélioré l'expérience utilisateur en garantissant une interaction fluide et sûre. L'incorporation de ces pratiques montre notre engagement envers la protection des utilisateurs et la préservation de leurs données contre les vulnérabilités courantes.

De plus, l'implémentation de ces méthodes permet de préparer l'application à des évolutions futures tout en facilitant la maintenance et l'extensibilité du code. En somme, ces choix techniques ne sont pas seulement des mesures de sécurité, mais également des piliers soutenant la pérennité de l'application dans un environnement numérique en constante évolution.



Tests

Introduction aux Tests Unitaires

Les tests unitaires sont essentiels pour vérifier la fonctionnalité des composants individuels d'un programme, permettant de détecter les bugs précocement et de maintenir la qualité du code.

Utilisation de Jest

Jest est un framework de tests JavaScript populaire pour son efficacité et sa simplicité, idéal pour les applications React et Node.js.

Test Unitaire sur la Fonction de Formatage de Date

Le test sur la fonction `formatDateCommentary` valide le traitement correct des dates valides, invalides, et celles avec différents fuseaux horaires, assurant la robustesse de notre fonctionnalité de formatage. Ci-dessous les résultats positifs dans notre terminal après avoir tapé la commande `npm run test` :

```
Amir NINACH@DESKTOP-NG8617U MINGW64 ~/Desktop/deadlinedrive/frontend
(dev)
$ npm test

> test
> jest

PASS  __tests__/formatDateCommentary.test.js
  formatDateCommentary
    ✓ should format the date string correctly (830 ms)
    ✓ should handle invalid date inputs gracefully (44 ms)
    ✓ should handle dates with different timezones consistently (1 ms)
  return go(f, seed, [])
}
```

Tests Manuels avec Postman

Introduction aux Tests Manuels

Les tests manuels sont cruciaux pour valider le comportement de notre API REST en conditions réelles. Nous avons utilisé Postman, un outil robuste pour envoyer des requêtes, observer les réponses pour assurer que notre API fonctionne correctement.

Procédure de Test avec Postman

1. **Configuration de l'environnement** : Nous avons préparé des environnements dans Postman pour gérer les paramètres essentiels comme les URLs et les tokens.
2. **Collection de Requêtes** : Les requêtes ont été organisées par fonctionnalités, permettant des tests systématiques et une documentation claire.
3. **Validation des Réponses** : Nous avons contrôlé les codes de statut, les corps de réponse, et les en-têtes pour chaque requête.





Déploiement

Déploiement sur Render

Déploiement de la partie Backend

Nous avons commencé par déployer la partie backend de notre application. Pour ce faire, nous avons utilisé la plateforme Render qui permet de déployer des services web de manière simple et efficace. Voici les étapes suivies pour le déploiement :

1. Création du Service Web :

- Nous avons créé un nouveau service de type "Web Service" sur Render.
- Nous avons sélectionné l'option "Node" pour le runtime, car notre backend est développé en JavaScript avec Node.js.
- Nous avons configuré le service pour qu'il pointe vers le dépôt GitHub contenant le code source du backend.

2. Configuration des Variables d'Environnement :

- Nous avons renseigné toutes les variables d'environnement nécessaires pour que la relation avec la base de données soit établie. Cela inclut des variables telles que `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, et `JWT_SECRET`.
- Ces variables permettent au backend de se connecter à notre base de données hébergée en ligne sur Always Data.

Déploiement de la partie Frontend

Ensuite, nous avons déployé la partie frontend de notre application qui est développée avec React.js. Les étapes suivies pour le déploiement du frontend sont les suivantes :

1. Création du Site Statique :

- Nous avons créé un nouveau service de type "Static Site" sur Render.
- Nous avons configuré ce service pour qu'il pointe vers le dépôt GitHub contenant le code source du frontend.

2. Paramétrage des Redirections :

- Pour s'assurer que notre application React fonctionne correctement en mode Single Page Application (SPA), nous avons configuré des règles de redirection. Cela permet de rediriger toutes les demandes vers le fichier index.html.
- Voici la configuration des redirections que nous avons mise en place (voir image ci-dessous) :

Redirect and Rewrite Rules

Add [Redirect or Rewrite Rules](#) to modify requests to your site. Use URL parameters to capture path segments and wildcards to redirect everything under a given path.

	Source	Destination	Action	
↑ ↓	/*	/index.html	Rewrite ▾	
			+ Add Rule	Save Changes



3. Configuration des Variables d'Environnement :

- Nous avons configuré l'URL du backend comme variable d'environnement dans la partie frontend. Cela permet à l'application React de faire des requêtes HTTP vers notre backend déployé.
- De même, nous avons renseigné l'URL du frontend dans la partie backend pour permettre la communication entre les deux services.

Configuration de la Base de Données

Notre base de données est hébergée en ligne sur Always Data. Nous avons veillé à ce que toutes les informations de connexion nécessaires soient correctement configurées dans les variables d'environnement de notre service backend sur Render. Cela inclut les détails de l'hôte, le port, l'utilisateur, le mot de passe, et le nom de la base de données.

Conclusion

Grâce à Render, nous avons pu déployer facilement notre application full-stack, comprenant un backend en Node.js et un frontend en React.js. La configuration des variables d'environnement et des redirections a été essentielle pour assurer une communication fluide entre les différentes parties de l'application et la base de données hébergée en ligne. (Documentation complète Page 66 et 67)

Recherches

a. Recherche en anglais

Pour développer et optimiser notre projet, j'ai consulté la documentation en anglais de plusieurs sites spécialisés. Sur le site [Cron-Job.org](https://cron-job.org), j'ai approfondi mes connaissances sur la planification des tâches automatisées, ce qui a amélioré notre système de notifications automatisées. J'ai également exploré [GitHub Heroicons](https://github.com/Heroicons) pour intégrer des icônes esthétiques et fonctionnelles à notre interface utilisateur. Enfin, la documentation sur [Tailwind CSS](https://tailwindcss.com) m'a aidé à l'installation et l'utilisation de ce framework CSS pour optimiser le design de notre application. Ces recherches m'ont permis d'apprendre de nouvelles technologies et de les mettre en pratique pour réaliser le développement de mes fonctionnalités au sein du projet Deadline Drive.

b. Veille technologique

Dans le cadre de ma veille technologique continue, je suis activement plusieurs chaînes YouTube et comptes Instagram influents dans le domaine du développement web. Parmi eux, la chaîne YouTube de [Chiny](https://www.youtube.com/channel/UCChiny) offre des tutoriels et des insights précieux sur les nouvelles tendances en technologie. De même, [Développeur Libre](https://www.youtube.com/channel/UCDevelopeurLibre) propose des conseils pratiques pour gérer sa carrière de développeur indépendant. Sur Instagram, je suis [Front-end Joe](https://www.instagram.com/Front-end-Joe), qui partage ses astuces et des ressources utiles pour les développeurs front-end. Ces sources enrichissent continuellement mes compétences et me permettent de rester à jour avec les dernières évolutions technologiques.



Conclusion

Ce projet m'a permis d'approfondir mes connaissances en React et de créer une application full stack intégrant une base de données et un backend robustes. Il a également renforcé mon esprit d'équipe et m'a permis de travailler dans des conditions réelles avec les contraintes d'un projet d'application web.

Pour l'avenir, les fonctionnalités à implémenter comprennent :

- L'amélioration du design global de l'application
- L'amélioration de l'expérience utilisateur avec l'ajout de notifications par e-mail à l'approche de leurs deadline
- La fonction de récupération de mot de passe avec la librairie Nodemailer
- L'amélioration de l'interface utilisateur avec l'ajout d'un avatar
- La modération des commentaires
- La possibilité de partager directement l'URL des citations sur Facebook, Twitter ou WhatsApp via le bouton de partage

Je suis très enthousiaste à l'idée de faire évoluer ce projet, en continuant à l'améliorer pour répondre aux besoins des utilisateurs ainsi que de le proposer au public.

Remerciements

Je tiens à exprimer ma profonde gratitude à mes formateurs, Adrien Sergent, Nicolas Malet, Salah et Jay, pour leur encadrement, leur patience et leur expertise tout au long de ma formation. Leur guidance a été essentielle à mon développement professionnel et à l'acquisition de compétences techniques.

Un merci spécial à tous mes camarades avec qui j'ai eu l'opportunité de collaborer, d'échanger et d'apprendre durant cette période enrichissante. En particulier, je souhaite remercier mes collaborateurs et amis sur le projet Deadline Drive : Faysoil, Jérôme et Calli. Travailler à vos côtés a été une expérience véritablement inspirante et éducative, remplie de défis stimulants et de réussites partagées.

Je suis également infiniment reconnaissant envers ma famille, mes amis et ma compagne Manon pour leur amour, leur soutien inconditionnel et leur encouragement constant. Leur présence et leur compréhension ont été des piliers dans les moments difficiles et une source de motivation pour poursuivre mes ambitions.

Enfin, je suis reconnaissant envers Simplon et toute son équipe pour avoir fourni un environnement d'apprentissage exceptionnel. Leur engagement envers l'excellence et l'innovation dans l'éducation technologique continue de permettre à de nombreux apprenants de réaliser leur potentiel.

Ces remerciements s'étendent à tous ceux qui ont contribué à mon parcours, directement ou indirectement.



Annexes

Autres Wireframes vers maquettes

Evolution du wireframe vers la maquette

Page Connexion (format mobile)

☒ Lorem ipsum



Log In

Email Address

Placeholder

Password

Placeholder 

It must be a combination of minimum 8 letters, numbers, and symbols.

☐ Remember me

Forgot Password?

Log In

No account yet? Sign Up

CompanyName @ 202X. All rights reserved.

Privacy policy Terms of service







Connexion

Adresse email

Votre email

Mot de passe

votre mot de passe 

Votre mot de passe doit faire un minimum de 6 caractères (numéros et spéciaux compris)

☐ Se Souvenir de moi

Mot de passe oublié ?

Connexion

Pas de compte ? Inscrivez vous

Deadline Drive@ 202X. Tous droits réservés

Politique de confidentialité Conditions Générales d'Utilisation



Page Inscription (format mobile)

☐ Lorem ipsum





Sign Up

First Name

Placeholder

Last Name

Placeholder

Email

Placeholder

Password

Placeholder



☐ Vestibulum faucibus odio vitae arcu auctor lectus

Button Text

Already have an account?

CompanyName @ 202X. All rights reserved.

[Privacy policy](#)

[Terms of service](#)







Inscription

Nom

Votre Nom

Prenom

Votre Prenom

Email

Votre email

Mot de passe

Votre mot de passe



☐ Se souvenir de moi

Inscription

Vous avez déjà un compte ? Connectez-vous

CompanyName @ 202X. All rights reserved.

[Politique de confidentialité](#)

[Conditions Générales d'Utilisation](#)



Page Récupération de mot de passe (format mobile)

☐ Lorem ipsum



Email adress

Envoyer





Votre Adresse email

Votre email...

Envoyer

Nous pouvons vous aider à réinitialiser votre mot de passe et vos informations de sécurité. Tout d'abord, entrez votre compte Microsoft et suivez les instructions ci-dessous.

CompanyName @ 202X. All rights reserved.

Privacy policy

Terms of service

Deadline Drive@ 202X. Tous droits réservés

Politique de confidentialité

Conditions Générales d'Utilisation

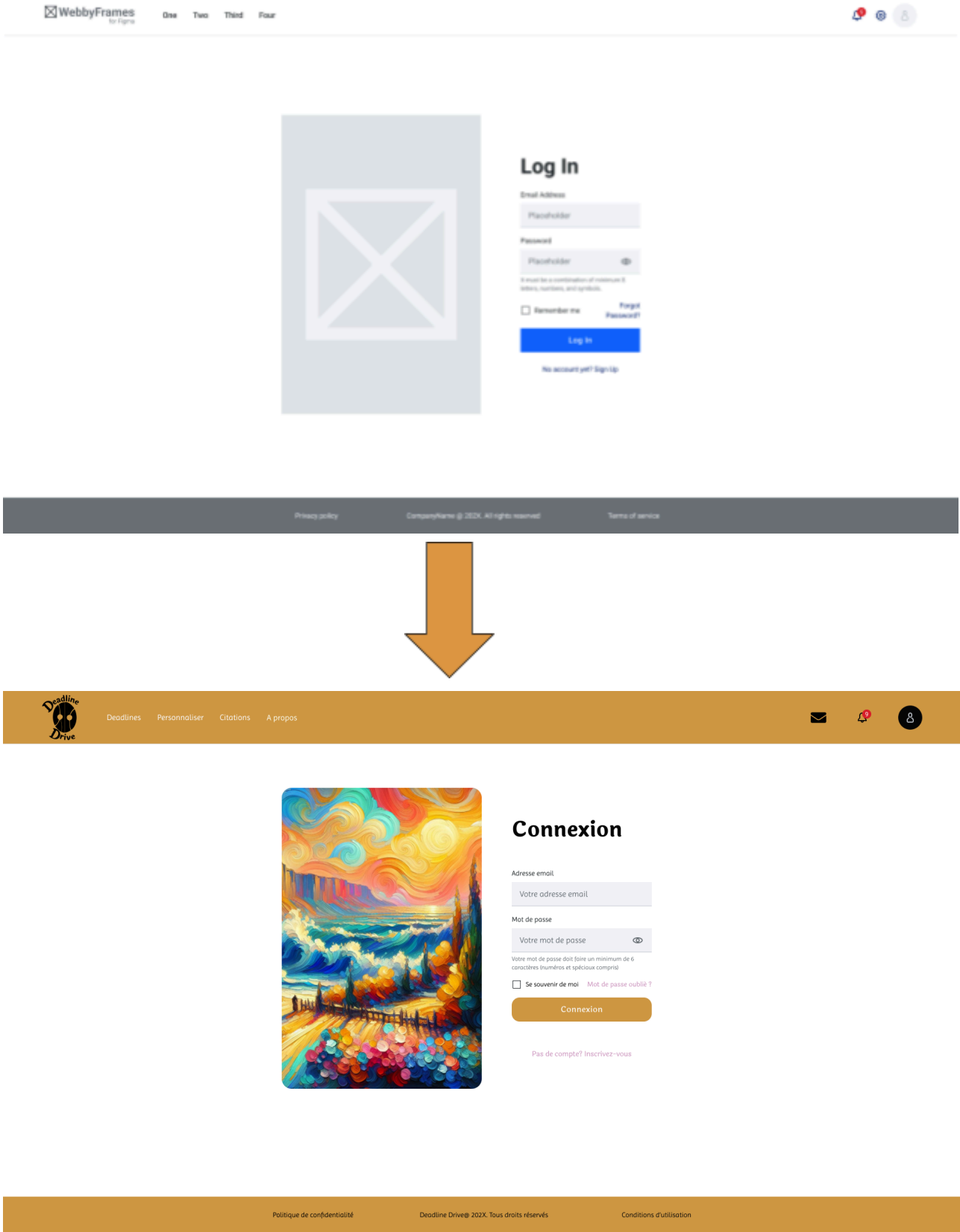


Modal Notifications (mobile)



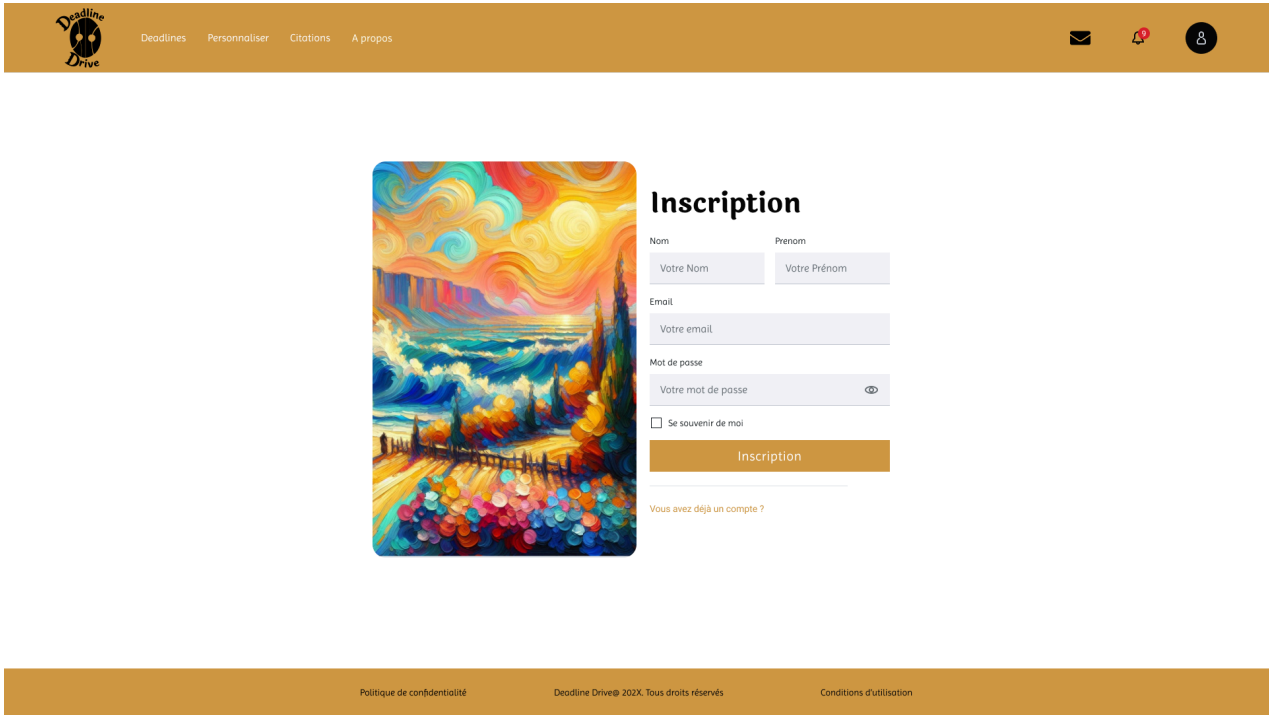
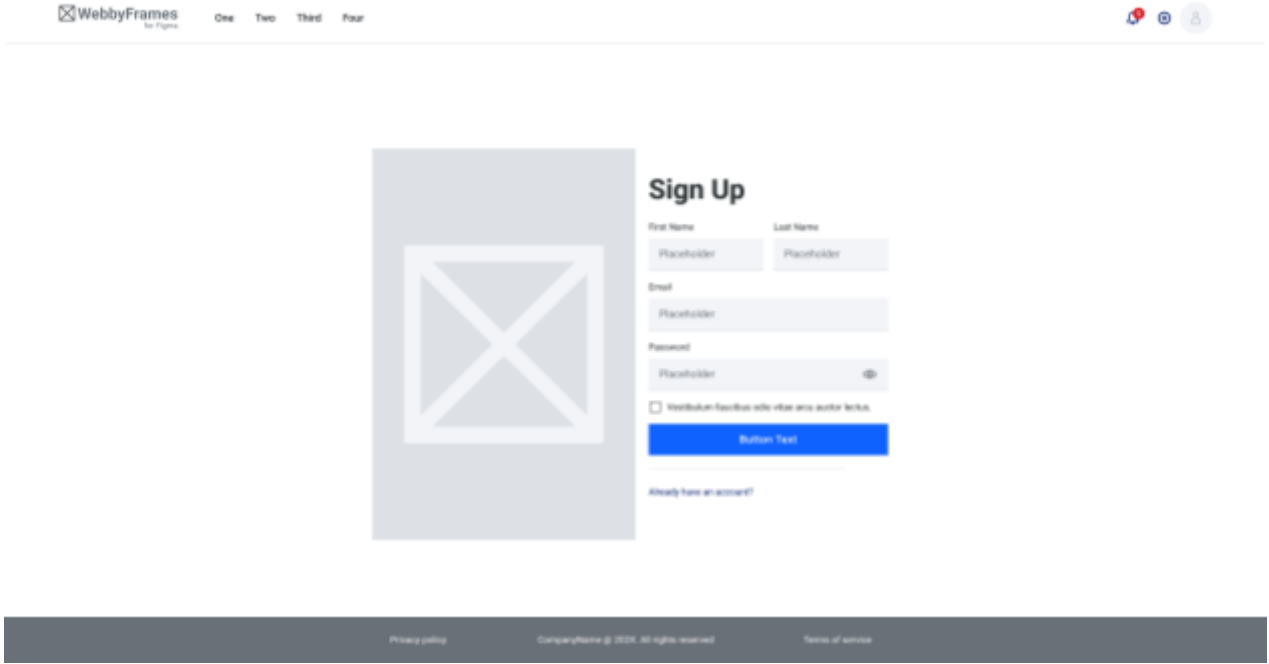


Page Connexion (format Desktop)





Page Inscription (Format Desktop)





WebbyFrames

to inspire

OneTwoThirdFour

Page A Propos (format Desktop)

NOTRE HISTOIRE

Purus sagittis fringilla arcu neque.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Bibendum amet at molestie mattis.

Rhoncus morbi et augue nec, in id ullamcorper at sit. Conditimentum sit nunc in eros scelerisque sed. Conmodo in viverra nunc, ullamcorper ut. Non, amet, aliquet scelerisque nullam sagittis, pulvinar. Fermentum scelerisque sit consectetur hac mi. Mollis leo eleifend ultricies purus laculis.

NOS CATEGORIES

Bibendum amet at molestie mattis.

Rhoncus morbi et augue nec, in id ullamcorper at sit. Conditimentum sit nunc in eros scelerisque sed. Conmodo in viverra nunc, ullamcorper ut. Non, amet, aliquet scelerisque nullam sagittis, pulvinar. Fermentum scelerisque sit consectetur hac mi. Mollis leo eleifend ultricies purus laculis.

OUR LOVELY TEAM

Kristin Watson

Sr. Product Designer

in

Kristin Watson

Sr. Product Designer

in

Kristin Watson

Sr. Product Designer

in

Kristin Watson

Sr. Product Designer

in

FAQ

Bibendum amet at molestie mattis.

Who should use the app?

+

What is included with my subscription?

+

How do I get paid?

+

Is my personal information safe?

+

How can we get in touch?

+

Privacy policy

Companyname © 2019. All rights reserved

Terms of service

Deadline

Personnalité

Citation

Appique

NOTRE HISTOIRE

Imaginez quatre étudiants en développement web, amoureux de leurs de café et de code, se faisant concurrence pour la programmation comme des chercheurs en quête de perfection. Un jour, alors qu'ils étaient sur le point de succomber à la tentation de programmer sur deux écrans. Pourquoi ne pas créer notre propre entreprise à la programmation ? Et ainsi naquit l'idée de DeadlineDrive - un projet né de leur désir de transformer leur lutte contre la procrastination en une aventure amusante et amusante pour tous. Avec l'ajoutage de café et de détermination, ils ont conçu une plateforme pour aider les autres à rester motivés, à atteindre leurs objectifs et à utiliser chaque petite victoire comme ils le feraient en train de compiler le leur projet du succès. Et voilà comment ces quatre étudiants ont transformé leur combat contre la procrastination en une quête épique pour la motivation.

NOS CATEGORIES

Auteur, Manga, Série, Film, Animé, Histoires, Poésie, Personnalité ou encore Sport

Découvrez l'inspiration à travers une multitude de perspectives avec notre étonnante variété de citations provenant de différentes catégories. Que vous soyez passionné par les grands auteurs, les recettes techniques des mangas et des séries, les mangas sophistiqués des séries et des films, les légendes de vieilles de l'histoire, les vers amusants de la poésie, les arrangements des personnalités influentes ou les exploits épiques du monde du sport, vous trouverez une source d'inspiration collable à vos goûts et à vos valeurs pour nourrir votre esprit et stimuler votre motivation chaque jour.

OUR LOVELY TEAM

WENJICH David

Developpeur Web Jr.

in

PETER Carl

Developpeur Web Jr.

in

GUINIO Jerome

Developpeur Web Jr.

in

CHALMERS Fayoull

Developpeur Web Jr.

in

FAQ

Comment puis-je m'inscrire sur DeadlineDrive ?

+

Qu'est-ce que le système de trending quotes sur DeadlineDrive ?

+

Comment puis-je choisir mes préférences de citations ?

+

Comment puis-je utiliser le calendrier interactif sur DeadlineDrive ?

+

Puis-je personnaliser les notifications que je reçois de DeadlineDrive ?

+

Politique de confidentialité

Deadline Drive 2019. Tous droits réservés.

Conditions d'utilisation

65



Documentation du Déploiement sur Render



Déploiement du Backend

Le déploiement du backend sur Render a commencé par la création d'un nouveau service web. Après avoir sélectionné le répertoire GitHub correspondant au projet (<https://github.com/KD63799/deadlinedrive>), le service a été nommé "deadlinedrive-api". Le répertoire racine spécifié était `./backend` et les commandes configurées pour la construction et le lancement étaient respectivement `npm install` et `npm run start`.

Les variables d'environnement nécessaires, définies dans `./backend/.env`, ont été saisies dans la section "Environment Variables" pour assurer la configuration correcte de l'environnement de production ainsi que la relation avec la base de données. Après le déploiement, nous avons validé l'accessibilité de l'API Rest en vérifiant la réponse en format JSON des données.

Déploiement du Frontend

Pour le frontend, la procédure était similaire mais adaptée pour un site statique. Après avoir cliqué sur "New" et sélectionné "Static Site", nous avons renseigné le même répertoire GitHub. Le service, intitulé "deadlinedrive", avait pour répertoire racine `./frontend`. Les commandes pour la construction étaient `npm install && npm run build`, avec le répertoire de publication défini sur `dist`.

Les variables d'environnement spécifiées dans `./frontend/.env` ont été configurées de manière identique à celles du backend pour assurer une cohérence entre les environnements de développement et de production. En résumé, il s'agit de la relation entre le front-end et le back-end.



Configuration des Redirections

Un ajustement crucial pour le fonctionnement correct du routing de l'application React était la configuration des redirections. Dans la section "Redirections" des paramètres Render, nous avons spécifié une règle de réécriture:

- Source: /*
- Destination: /index.html
- Type: Rewrite

Cette configuration garantissait que toutes les requêtes étaient correctement dirigées vers le fichier index.html, permettant ainsi à React Router de gérer le routage côté client. En effet, les applications React.js sont des SPA (single page application), cet ajustement est vital pour le bon fonctionnement du site.

Validation Finale

Après avoir configuré les redirections et déployé le frontend, une vérification finale a été réalisée pour s'assurer que toutes les fonctionnalités de l'application étaient opérationnelles. Cette étape comprenait des tests de navigation dans l'application pour confirmer que le routage et les fonctionnalités clés fonctionnaient comme prévu.

Ce processus de déploiement sur Render a non seulement simplifié la mise en ligne de notre application mais a aussi permis une intégration fluide entre le backend et le frontend, garantissant une expérience utilisateur cohérente et performante.



API Deadline Drive

Description

Bienvenue dans la documentation de l'API RESTful de Deadline Drive. Cette API permet de gérer les utilisateurs, les citations, les commentaires, les catégories, les deadlines, et les tâches. Conçue selon les normes REST, elle est accessible via des requêtes HTTP standard, avec les données retournées en format JSON pour une intégration facile dans toute application ou site web. Dans les sections suivantes, nous détaillerons les différentes fonctionnalités et routes de l'API, ainsi que des exemples de requêtes et de réponses.

Endpoints

Home page:

- **GET** localhost:5000/

Users:

- **Browse:** GET
localhost:5000/users
- **Read:** GET
localhost:5000/users/:id
- **Edit:** PUT
localhost:5000/users/:id
- **Add:** POST
localhost:5000/users
- **Deletion:** DELETE
localhost:5000/users/:id

Quotes:

- **Browse:** GET
localhost:5000/quotes
- **Read:** GET
localhost:5000/quotes/:id
- **Edit:** PUT
localhost:5000/quotes/:id
- **Add:** POST
localhost:5000/quotes
- **Deletion:** DELETE
localhost:5000/quotes/:id

Notifications

- **Browse** GET
localhost:5000/notifications
- **Read** GET
localhost:5000/notifications/:id
- **Create** POST
localhost:5000/notifications
- **Mark** PUT
localhost:5000/notifications/:id/read
- **Delete** DELETE
localhost:5000/notifications/:id
- **Find** GET
localhost:5000/notifications/user/:userId

Deadlines:

- **Browse:** GET
localhost:5000/deadlines
- **Read:** GET
localhost:5000/deadlines/:id
- **Edit:** PUT
localhost:5000/deadlines/:id
- **Add:** POST
localhost:5000/deadlines
- **Deletion:** DELETE
localhost:5000/deadlines/:id

Quote-Categories:

- **Browse:** GET
localhost:5000/quote_categories
- **Read:** GET
localhost:5000/quote_categories/:id
- **Edit:** PUT
localhost:5000/quote_categories/:id
- **Add:** POST
localhost:5000/quote_categories
- **Deletion:** DELETE
localhost:5000/quote_categories/:id

Favorites

- **Add:** POST
localhost:5000/favorites
- **Get:** GET
localhost:5000/favorites/:id
- **Remove:** DELETE
localhost:5000/favorites/:quoteId

Comments:

- **Browse:** GET
localhost:5000/comments
- **Read:** GET
localhost:5000/comments/:id
- **Edit:** PUT
localhost:5000/comments/:id
- **Add:** POST
localhost:5000/comments
- **Deletion:** DELETE
localhost:5000/comments/:id

Categories:

- **Browse:** GET
localhost:5000/categories
- **Read:** GET
localhost:5000/categories/:id
- **Edit:** PUT
localhost:5000/categories/:id
- **Add:** POST
localhost:5000/categories
- **Deletion:** DELETE
localhost:5000/categories/:id

Tasks:

- **Browse:** GET
localhost:5000/tasks
- **Read:** GET
localhost:5000/tasks/:id
- **Edit:** PUT
localhost:5000/tasks/:id
- **Add:** POST
localhost:5000/tasks
- **Deletion:** DELETE
localhost:5000/tasks/:id



Exemple Utilisateurs

Récupérer tous les utilisateurs

Requête :

GET localhost:5000/users

Réponse :

```
{
  "users": [
    {
      "id": 1,
      "firstname": "Adrien",
      "lastname": "Sergent",
      "email": "as@example.com",
      "password": "hashed_password",
    },
    {
      "id": 2,
      "firstname": "Jane",
      "lastname": "Smith",
      "email": "janesmith@example.com",
      "password": "hashed_password",
    }
  ]
}
```

Recuperer un utilisateur par son ID

Requête :

GET localhost:5000/users/:id

Reponse :

```
{
  "id": 1,
  "firstname": "John",
  "lastname": "Doe",
  "email": "johndoe@example.com",
  "password": "hashed_password",
}
```

Ajouter un utilisateur

Requête :

POST localhost:5000/users

Body :

```
{
  "firstname": "John",
  "lastname": "Doe",
  "email": "johndoe@email.com",
  "password": "password_hash",
}
```

markdown

Copier le code

Exemple Utilisateurs

Recuperer tous les utilisateurs

==Requête :==

GET 'localhost:5000/users'

==Reponse :==

'''json

```
{
  "users": [
    {
      "id": 1,
      "firstname": "Adrien",
      "lastname": "Sergent",
      "email": "as@example.com",
      "password": "hashed_password",
    },
    {
      "id": 2,
      "firstname": "Jane",
      "lastname": "Smith",
      "email": "janesmith@example.com",
      "password": "hashed_password",
    }
  ]
}
```



Documentation des EndPoint

Recuperer un utilisateur par son ID Requête : GET localhost:5000/users/:id Reponse : json Copier le code <pre>{ "id": 1, "firstname": "John", "lastname": "Doe", "email": "johndoe@example.com", "password": "hashed_password", }</pre>	Mise à jour d'un utilisateur Requête : PUT localhost:5000/users/:id Body : <pre>{ "firstname": "John", "lastname": "Doe", "email": "johndoe@example.com", "password": "mysecretpassword", }</pre> Reponse HTTP/1.1 200 OK : <pre>{ "id": 42, "firstname": "John", "lastname": "Doe", "email": "johndoe@example.com", "password": "mysecretpassword", }</pre>
Ajouter un utilisateur Requête : POST localhost:5000/users Body : json Copier le code <pre>{ "firstname": "John", "lastname": "Doe", "email": "johndoe@example.com", "password": "password_hash", }</pre>	Effacer un utilisateur Requête : DELETE localhost:5000/users/:id Reponse : HTTP/1.1 200 OK <pre>{ "message": "Utilisateur supprimé avec succès." }</pre>
Mise à jour d'un utilisateur Requête : PUT localhost:5000/users/:id Body : <pre>{ "firstname": "John", "lastname": "Doe", "email": "johndoe@example.com", "password": "mysecretpassword", }</pre>	

Fin de la documentation des EndPoint

Ressources externes

<https://github.com/KD63799/deadlinedrive> Readme.md en anglais (projet global)
<https://github.com/KD63799/deadlinedrive/blob/dev/backend/API.md> Api.md (Documentaiton
Api Restful du projet)
<https://github.com/KD63799/deadlinedrive/blob/main/backend/README.md> README.md
Documentation du Backend du projet
<https://docs.cron-job.org/> Documentation Cron
<https://github.com/tailwindlabs/heroicons> Documentation Hero Icons
<https://tailwindcss.com/docs/installation> Documentation Tailwind CSS
<https://www.youtube.com/@MohamedChiny> Chaîne Youtube Chiny
<https://www.youtube.com/c/D%C3%A9veloppeurLibre> Chaîne Youtube Développeur Libre
<https://www.instagram.com/frontendjoe/> Page Instagram Front-end Joe
<https://help.alwaysdata.com/fr/sauvegardes/> Sauvegarde always data
<https://deadlinedrive-p8op.onrender.com/> URL du site déployé sur render

Dossier de projet
Deadline Drive

www.deadlinedrive.com

